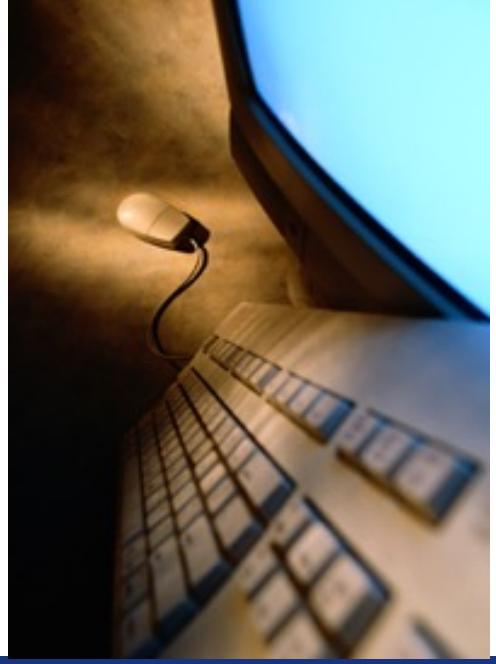
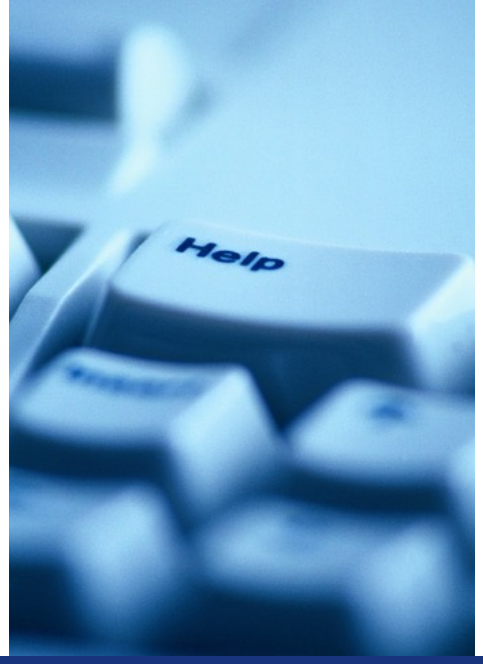



操作系统原理

Operating Systems Principles

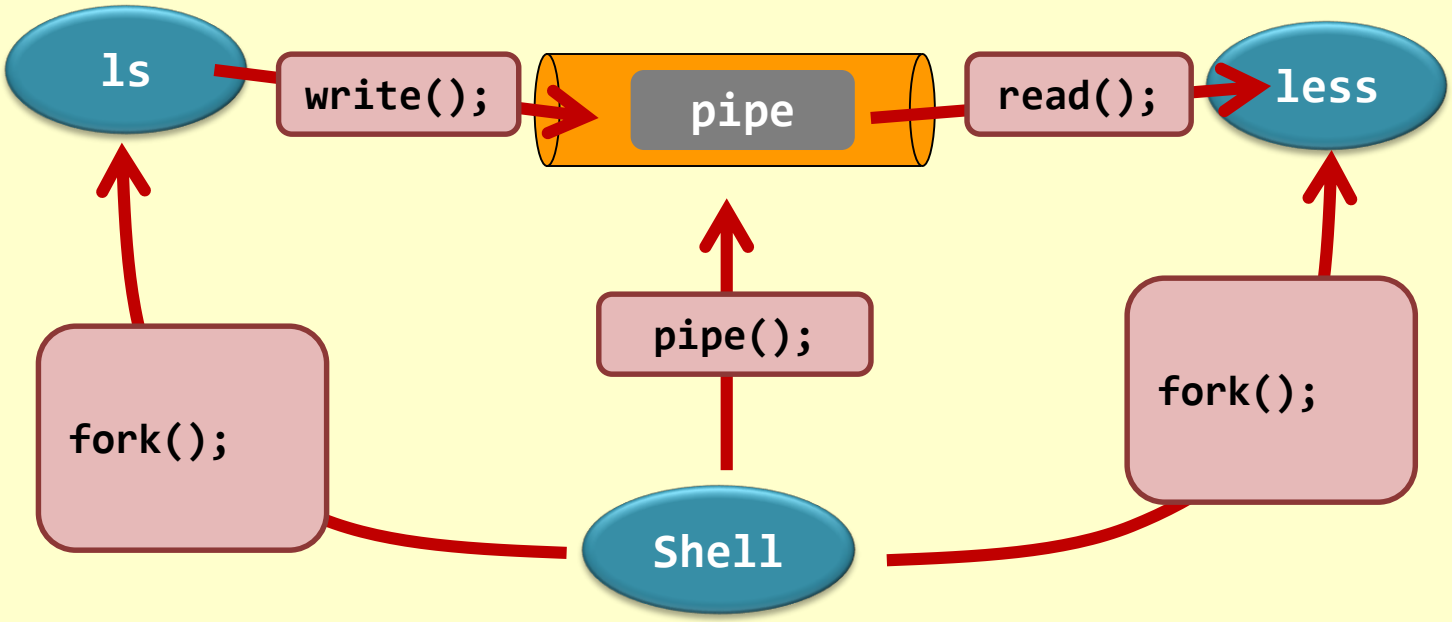
陈鹏飞
计算机学院

第七讲 —同步

课堂练习

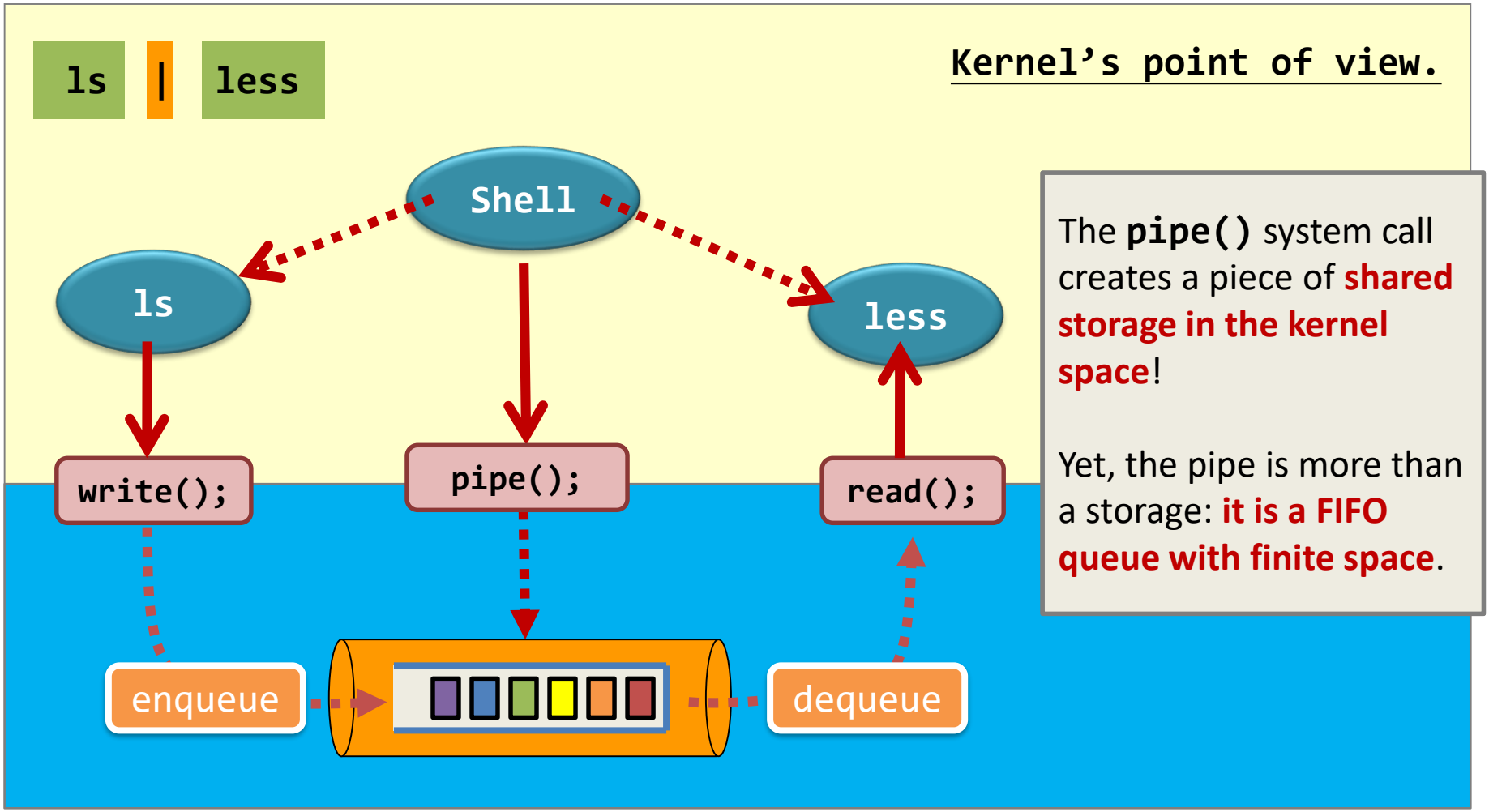
`ls | less`

Programmer's point of view.



Shell 中的"|"是匿名管道还是命名管道?

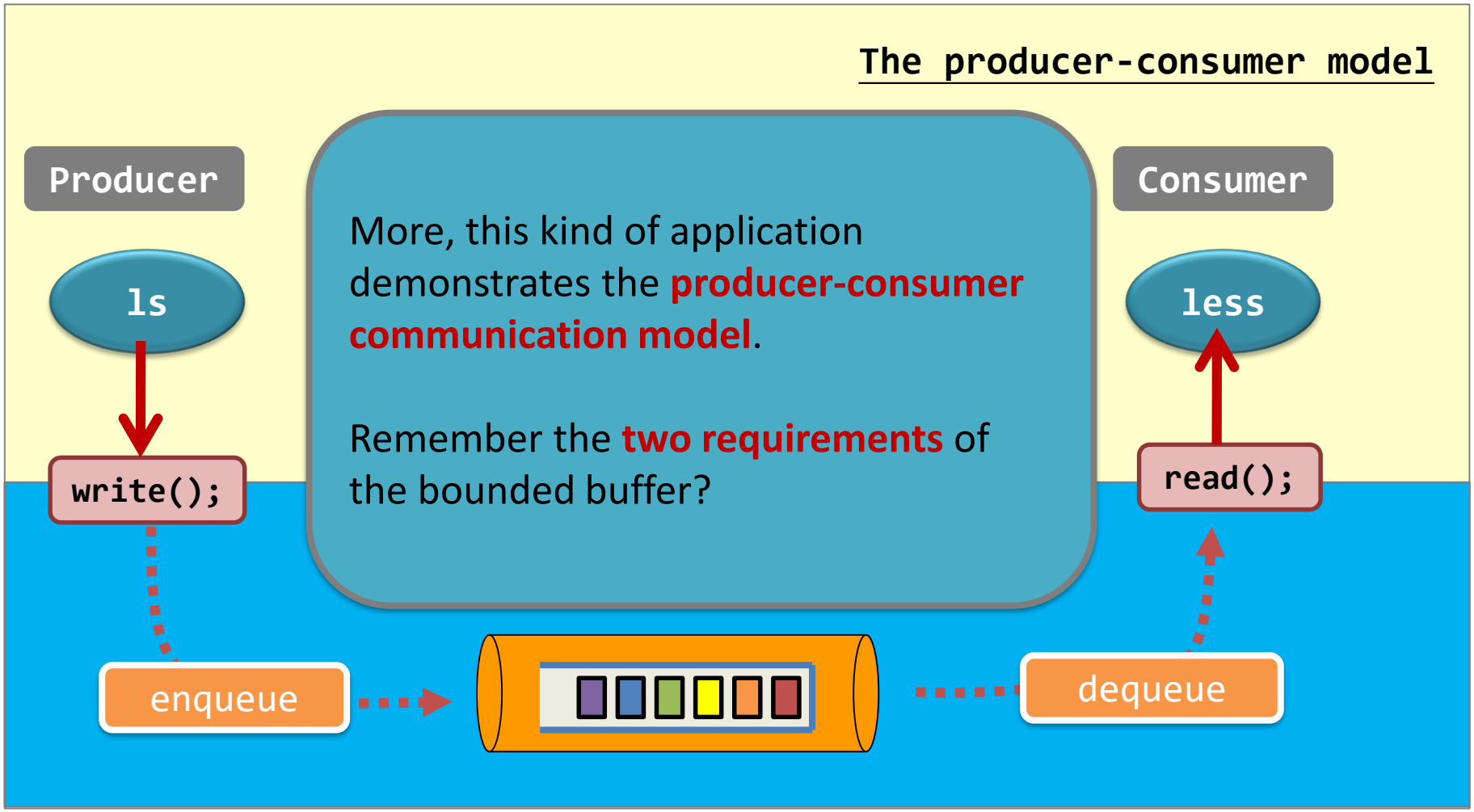
课堂练习

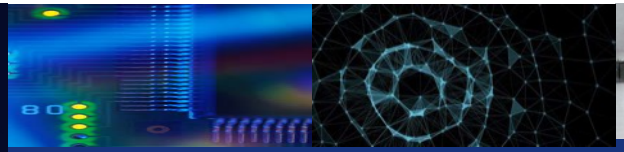


The **pipe()** system call creates a piece of **shared storage in the kernel space**!

Yet, the pipe is more than a storage: **it is a FIFO queue with finite space**.

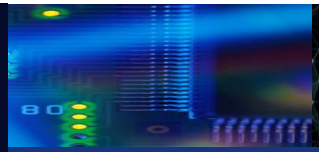
课堂练习


The producer-consumer model
Producer
Consumer
ls
less
write();
read();
More, this kind of application demonstrates the producer-consumer communication model.
Remember the two requirements of the bounded buffer?
enqueue
dequeue



概述

- 背景
- 临界区问题
- Peterson解决方案
- 硬件同步
- 互斥锁
- 信号量
- 管程
- Liveness（活性）
- 评价
- 经典同步问题—读者-写者，哲学家就餐、生产者消费者；
- 不同操作系统中的同步实现；



目标

- 介绍临界区问题并举例说明竞争条件；
- 说明使用比较和交换操作以及原子变量解决临界区问题的硬件解决方案；
- 如何使用互斥锁、信号量、管程和条件变量来解决临界区问题(软件解决方案)；
- 分析进程同步的多个经典问题；
- 探讨解决进程同步问题的多个工具；

背景-多进程

- 操作系统设计的核心问题是进程和线程的管理：
  - 多道程序设计；
  - 多处理器技术；
  - 分布式处理技术；

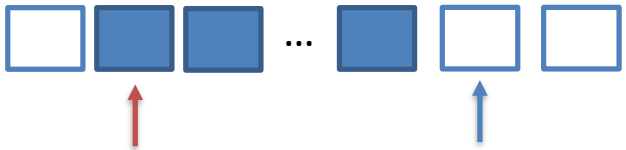
背景-进程执行

- 进程可以并发甚至并行执行
  - 可随时中断，部分完成执行
- 对共享数据的并发访问可能导致数据不一致
- 维护数据一致性需要确保协作进程有序执行的机制
- 考虑使用由生产者和消费者同时更新的计数器的有界缓冲区问题时的问题，这导致了竞争状态。

生产者-消费者问题解决方案

```
#define BUFFER_SIZE 10
typedef struct {
   . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Shared memory by producer & consumer processes



out (consumer) in (producer)

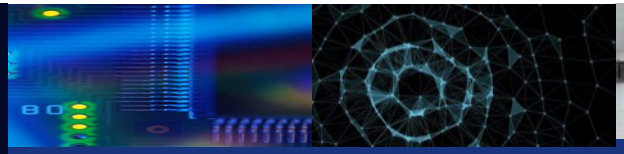
Only allows BUFFER_SIZE-1 items at the same time. Why?

```
item next_produced;
while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
}
```

Producer

```
item next_consumed;
while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item */
}
```

Consumer



生产者-消费者问题解决方案

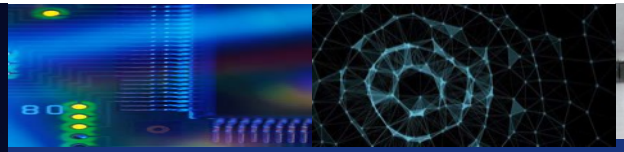
引入counter，初始化为0，可以解决最大为Buffer_size -1的问题；但是多个进程访问count产生竞争。

```
while (true) {
     /* produce an item in next_produced */

     while (count == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

生产者

```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;

     /* consume the item in next_consumed */
}
```

消费者



生产者-消费者问题解决方案

$register_1 = \texttt{count}$
$register_1 = register_1 + 1$
$\texttt{count} = register_1$

$register_2 = \texttt{count}$
$register_2 = register_2 - 1$
$\texttt{count} = register_2$

Counter ++ 操作

Counter -- 操作

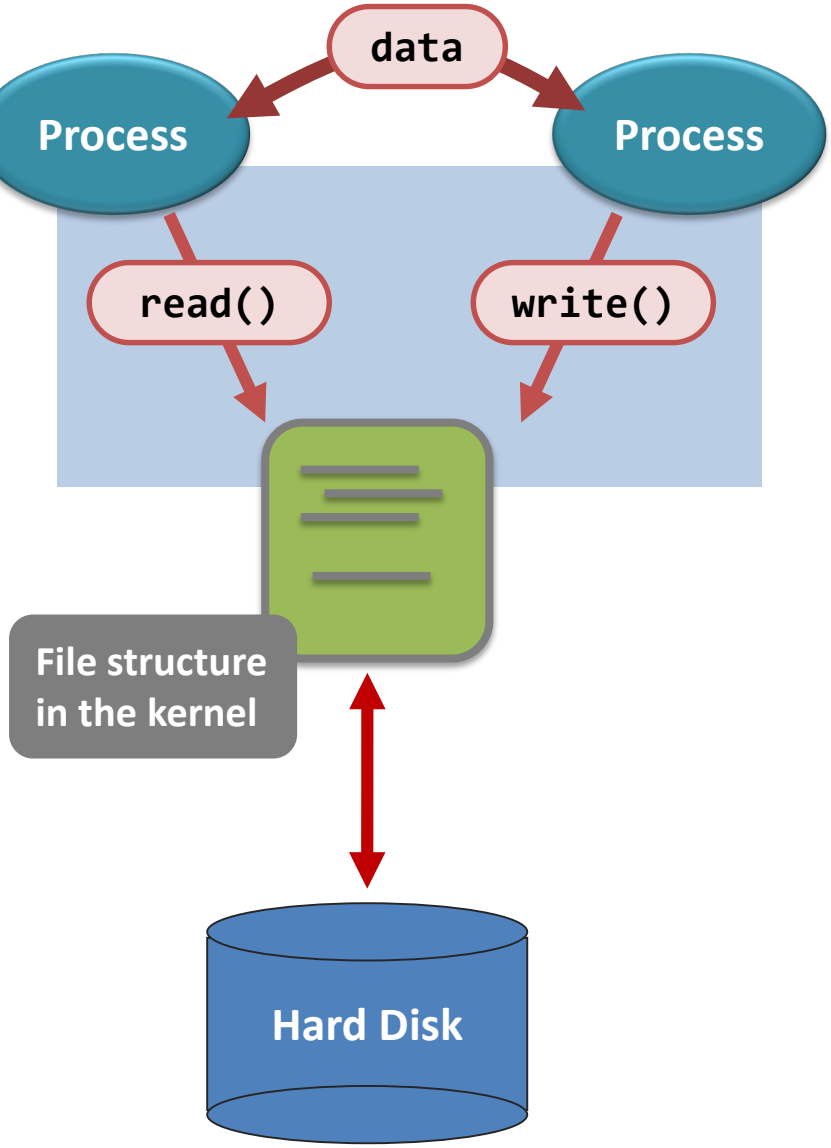
T_0:	*producer*	execute	$register_1 = \texttt{count}$	$\{register_1 = 5\}$
T_1:	*producer*	execute	$register_1 = register_1 + 1$	$\{register_1 = 6\}$
T_2:	*consumer*	execute	$register_2 = \texttt{count}$	$\{register_2 = 5\}$
T_3:	*consumer*	execute	$register_2 = register_2 - 1$	$\{register_2 = 4\}$
T_4:	*producer*	execute	$\texttt{count} = register_1$	$\{count = 6\}$
T_5:	*consumer*	execute	$\texttt{count} = register_2$	$\{count = 4\}$

一种执行过程

因为允许两个进程并发操作变量counter，所以得不到正确的状态。

多进程共享内存问题

- Pipe is implemented with the thought that **there may be more than one process accessing it *"at the same time"***
- For shared memory and files, **concurrent access may yield unpredictable outcomes**



多进程共享内存问题

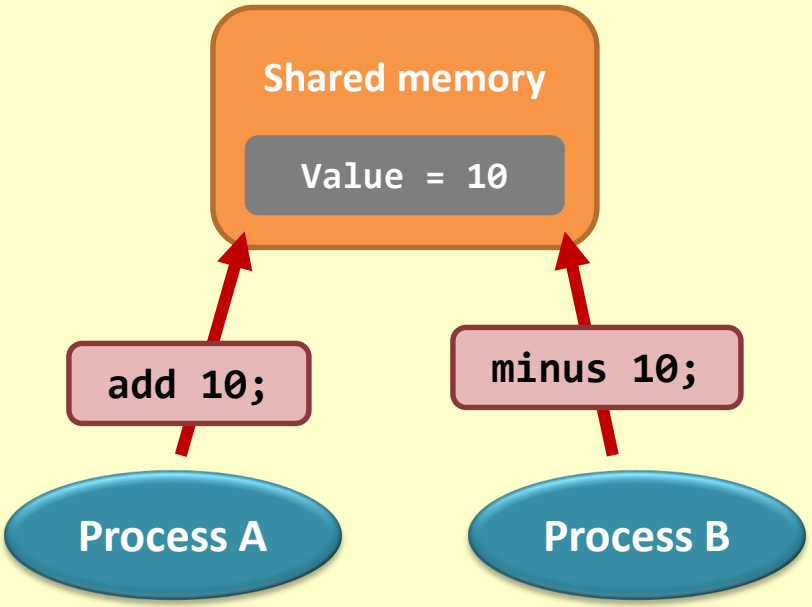
High-level language for Program A

```
1  attach to the shared memory X;
2  add 10 to X;
3  exit;
```

High-level language for Program B

```
1  attach to the shared memory X;
2  minus 10 to X;
3  exit;
```

The Scenario



Guess what the final result should be?

It may be 10, 0 or 20, can you believe it?

多进程共享内存问题

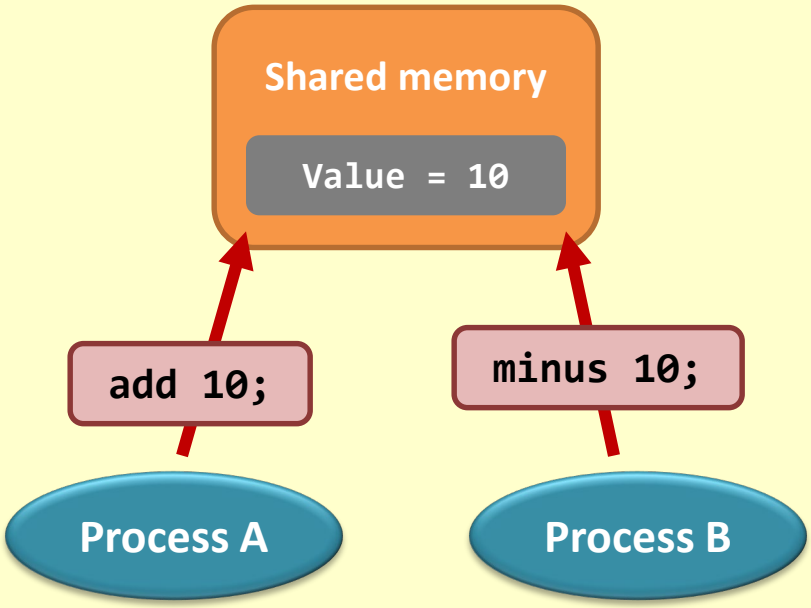
High-level language for Program A

```
1   attach to the shared memory X;
2   add 10 to X;
3   exit;
```

High-level language for Program B

```
1   attach to the shared memory X;
2   minus 10 to X;
3   exit;
```

The Scenario



Remember the flow of executing a program and the system hierarchy?

多进程共享内存问题

High-level language for Program A

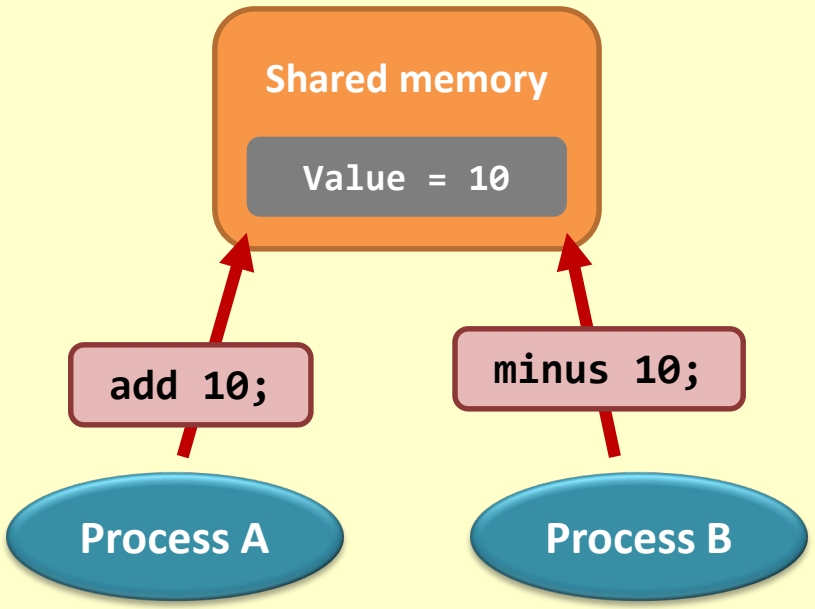
```
1  attach to the shared memory X;
2  add 10 to X;
3  exit;
```

This operation is not atomic

Partial low-level language for Program A

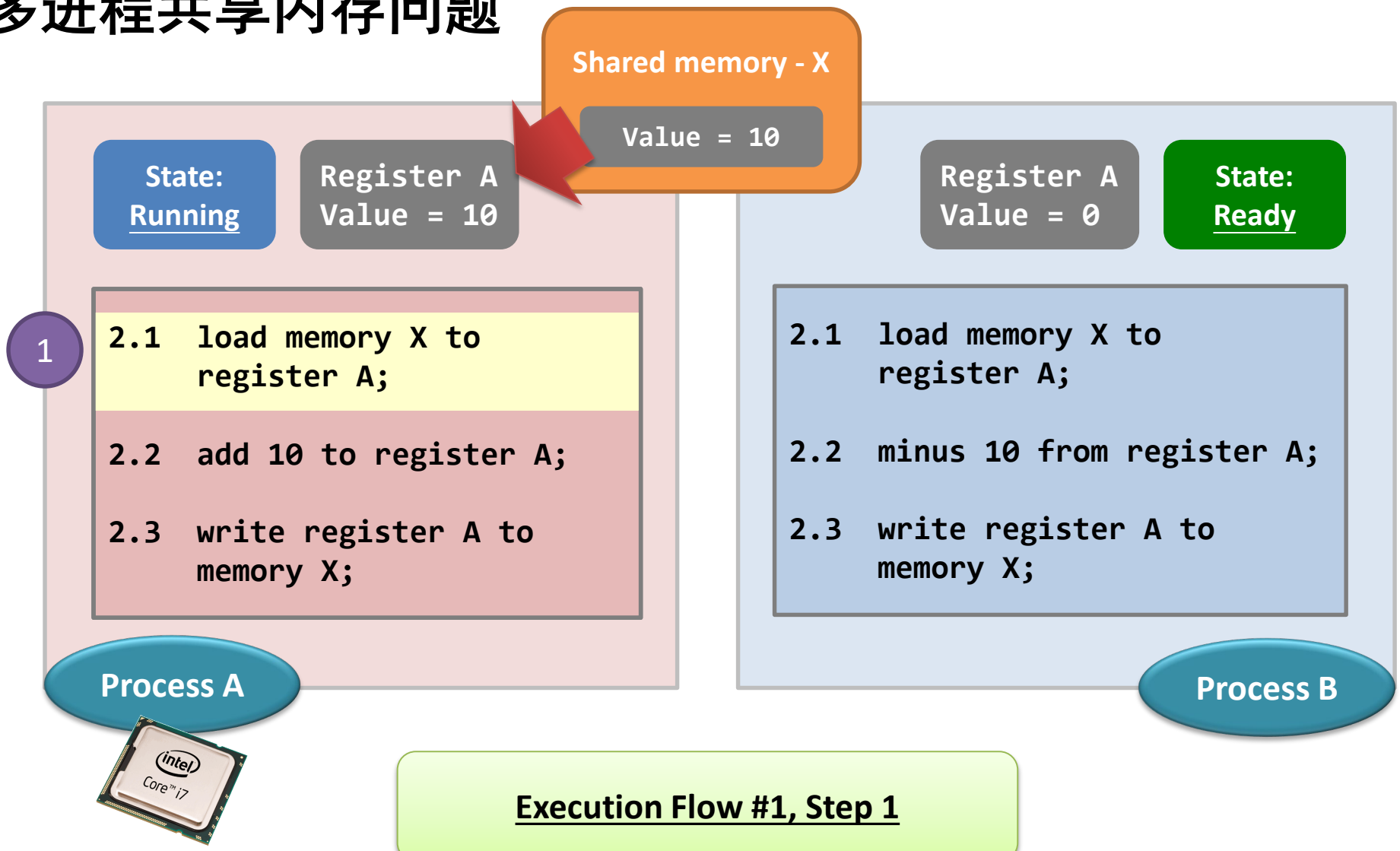
```
1    attach to the shared memory X;
......
2.1  load memory X to register A;
2.2  add 10 to register A;
2.3  write register A to memory X;
......
3    exit;
```

The Scenario



Guess what? This code block is evil!

多进程共享内存问题


Shared memory - X
Value = 10
State: Running
Register A Value = 10
1
2.1 load memory X to register A;
2.2 add 10 to register A;
2.3 write register A to memory X;
Process A
Register A Value = 0
State: Ready
2.1 load memory X to register A;
2.2 minus 10 from register A;
2.3 write register A to memory X;
Process B
Execution Flow #1, Step 1

多进程共享内存问题

Shared memory - X

Value = 10

Process A

State: Running

Register A Value = 20

1 `2.1 load memory X to register A;`

2 `2.2 add 10 to register A;`

`2.3 write register A to memory X;`

Process B

Register A Value = 0

State: Ready

`2.1 load memory X to register A;`

`2.2 minus 10 from register A;`

`2.3 write register A to memory X;`

Execution Flow #1, Step 2

多进程共享内存问题

Shared memory - X

Value = 20

State: Running

Register A Value = 20

Register A Value = 0

State: Ready

1 2.1 load memory X to register A;

2 2.2 add 10 to register A;

3 2.3 write register A to memory X;

2.1 load memory X to register A;

2.2 minus 10 from register A;

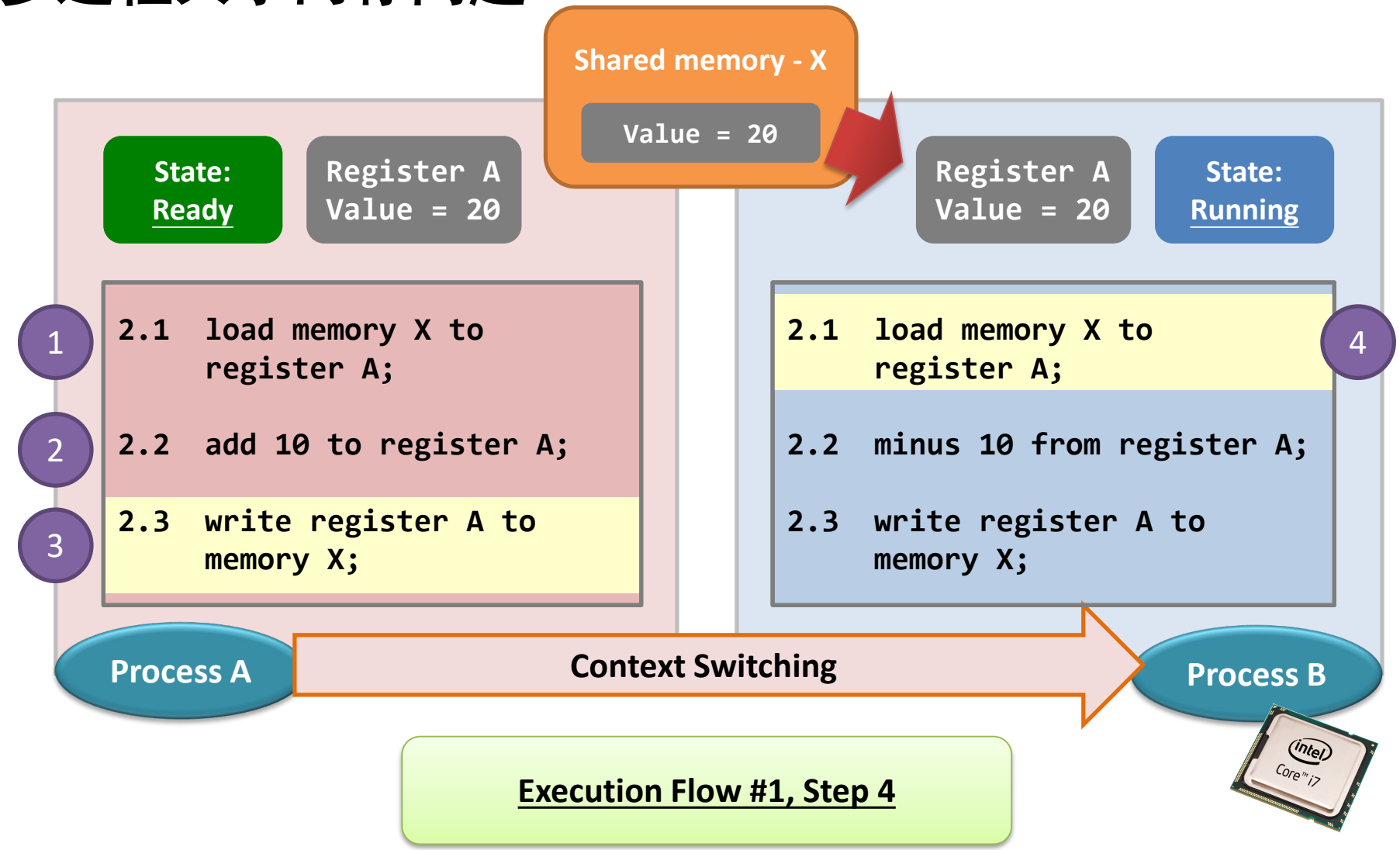
2.3 write register A to memory X;

Process A

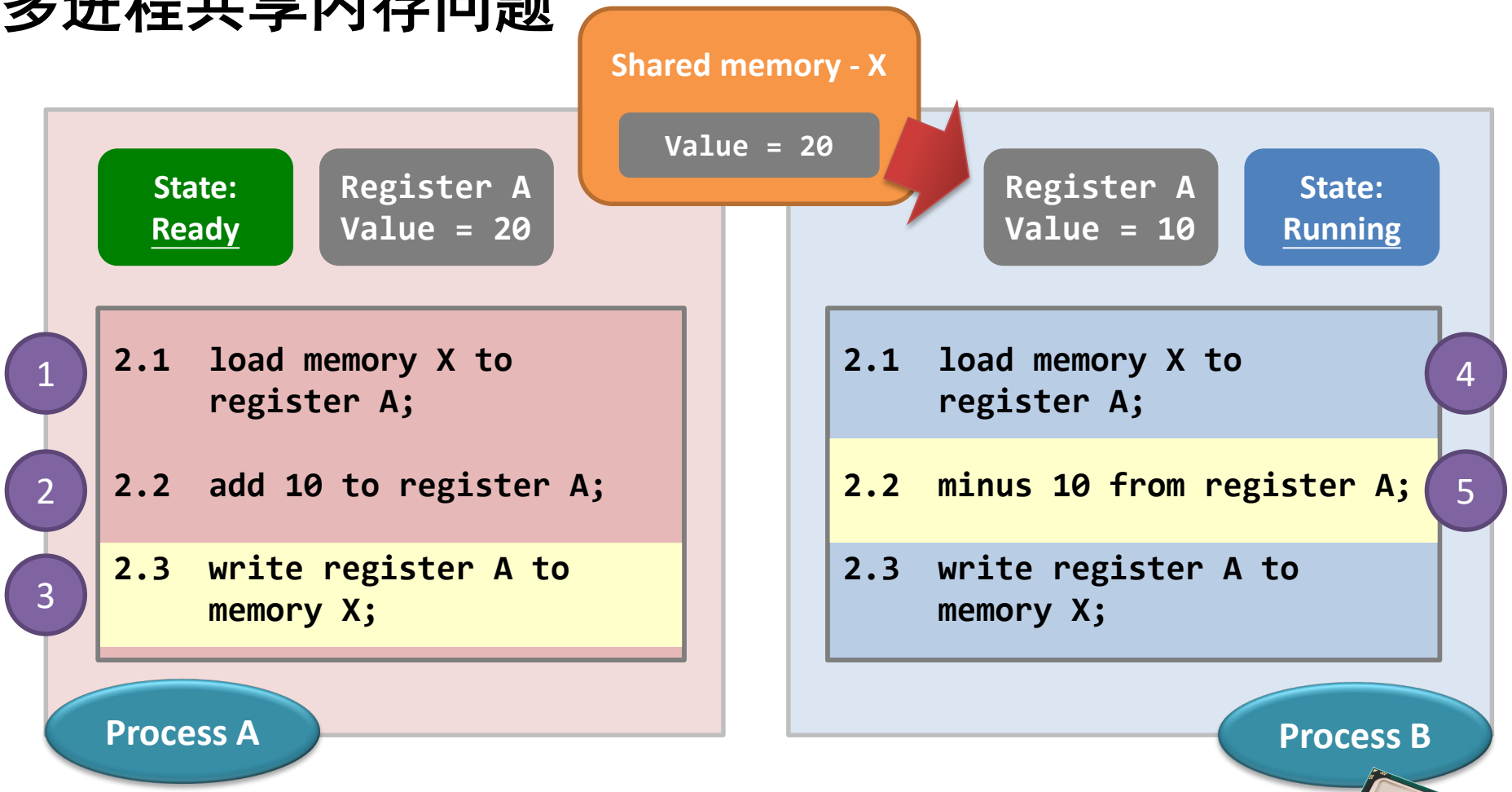
Process B

Execution Flow #1, Step 3

多进程共享内存问题

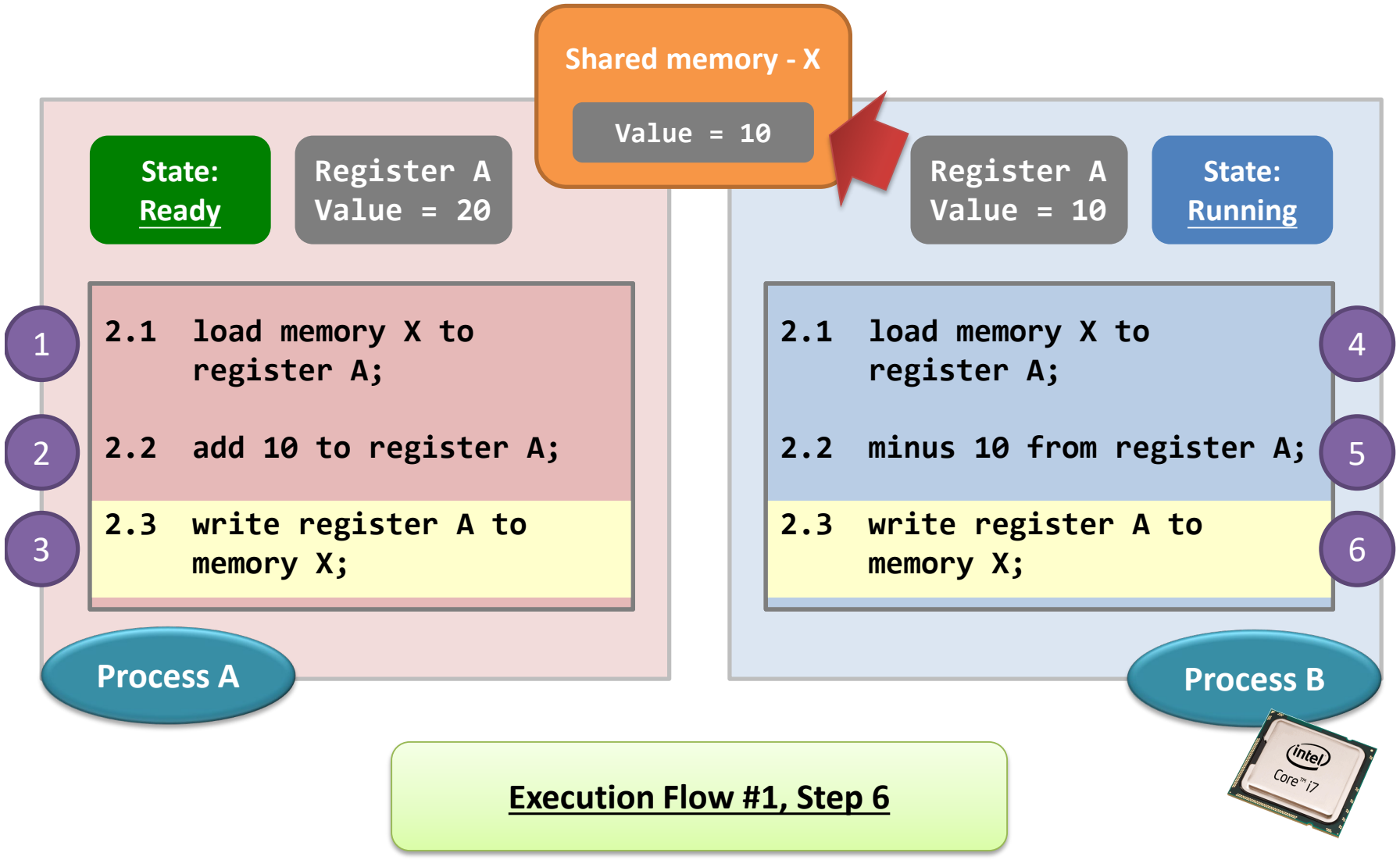

Shared memory - X
Value = 20
State: Ready
Register A Value = 20
Register A Value = 20
State: Running
1
2.1 load memory X to register A;
2
2.2 add 10 to register A;
3
2.3 write register A to memory X;
4
2.1 load memory X to register A;
2.2 minus 10 from register A;
2.3 write register A to memory X;
Process A
Context Switching
Process B
Execution Flow #1, Step 4

多进程共享内存问题

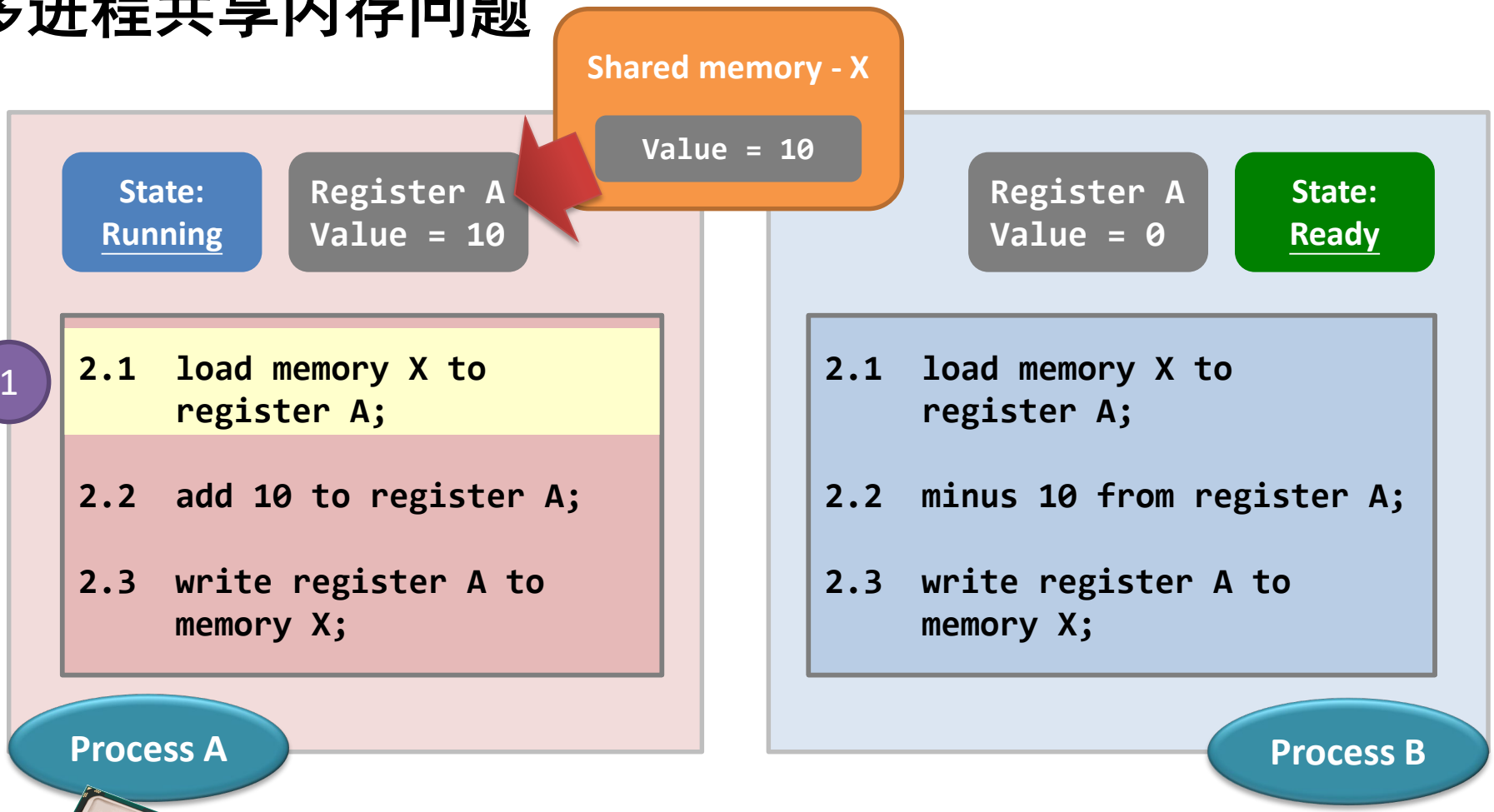


Execution Flow #1, Step 5

多进程共享内存问题

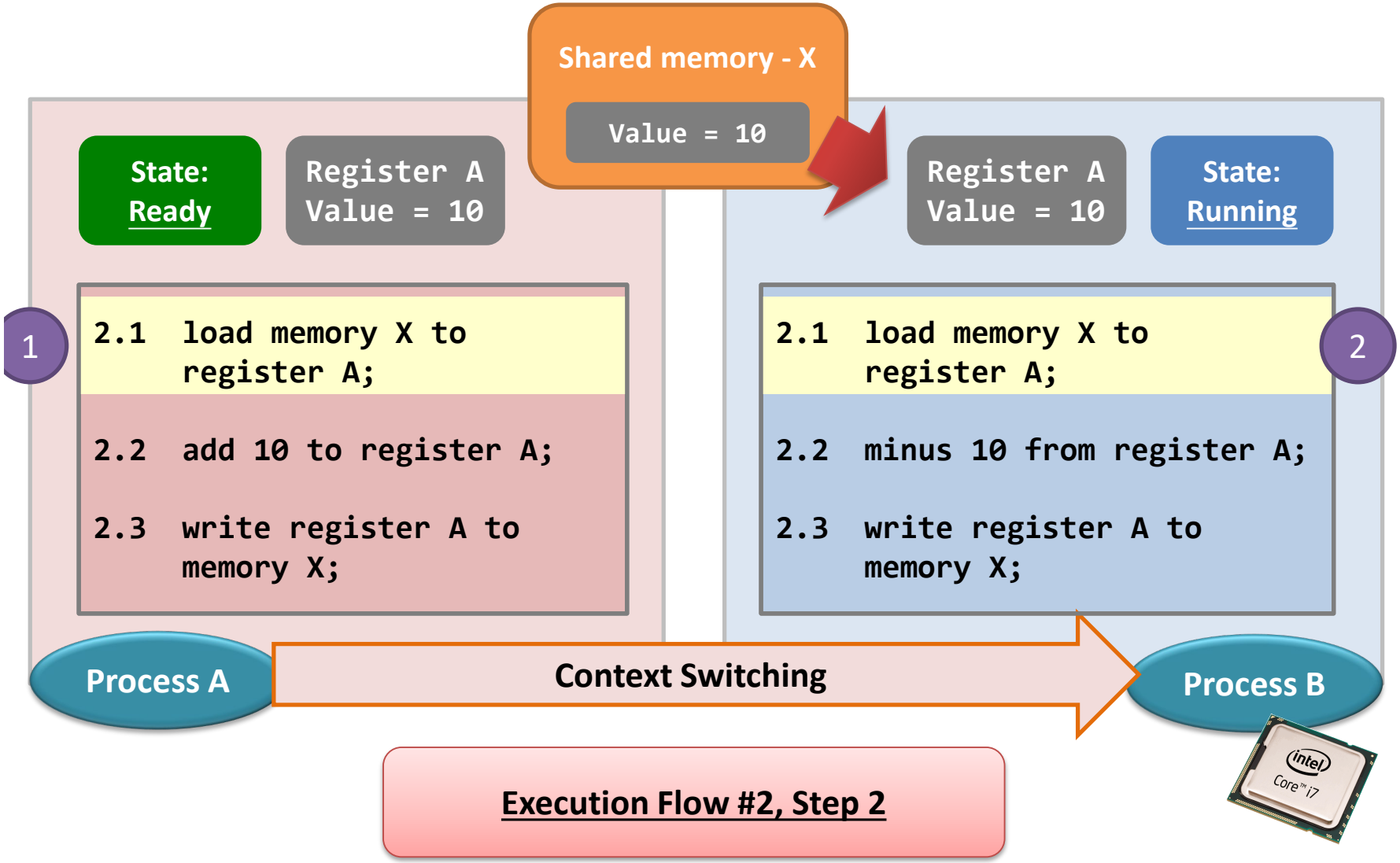

Shared memory - X
Value = 10
State: Ready
Register A Value = 20
2.1 load memory X to register A;
2.2 add 10 to register A;
2.3 write register A to memory X;
1
2
3
Process A
Register A Value = 10
State: Running
2.1 load memory X to register A;
2.2 minus 10 from register A;
2.3 write register A to memory X;
4
5
6
Process B
Execution Flow #1, Step 6

多进程共享内存问题

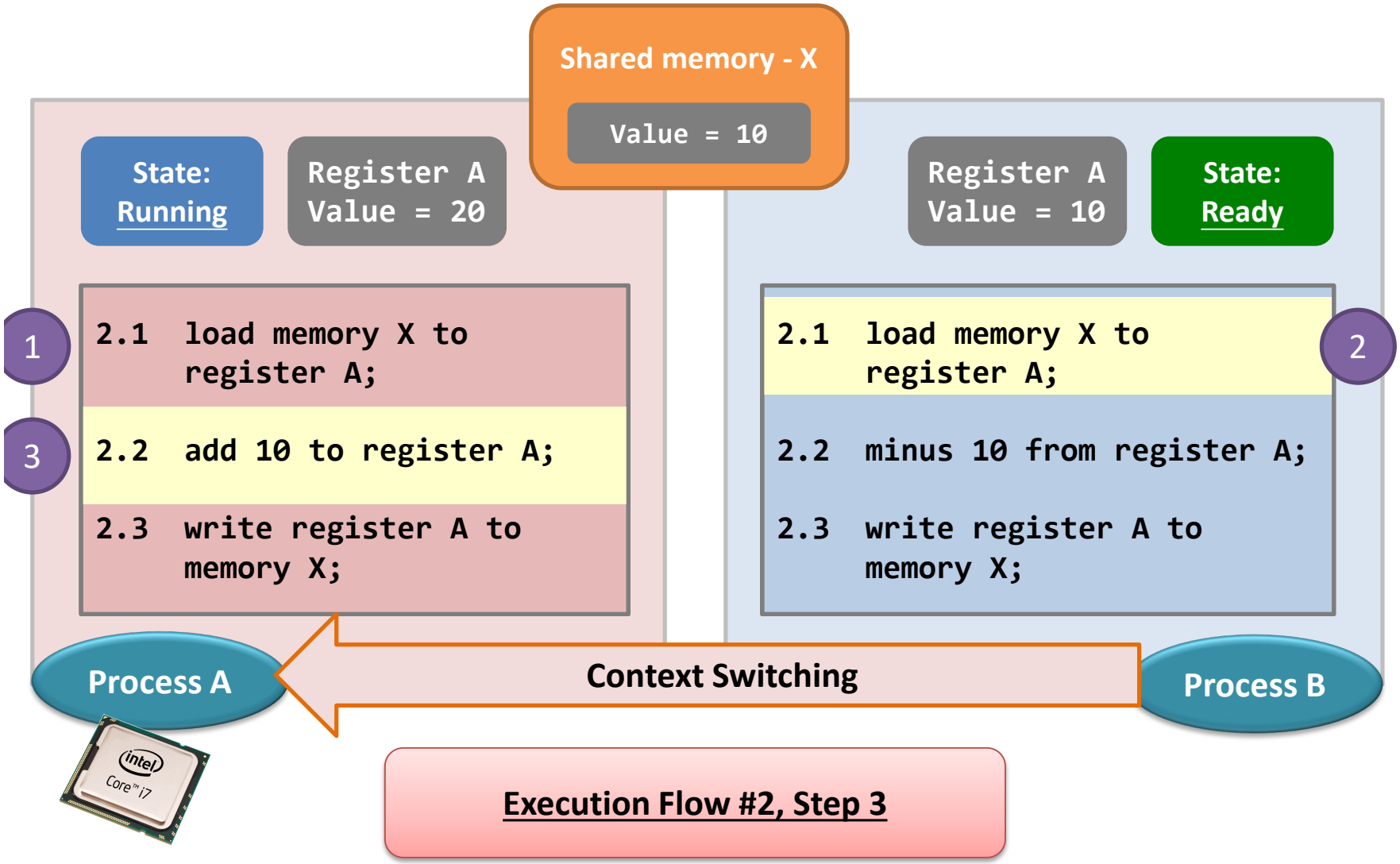


Execution Flow #2, Step 1

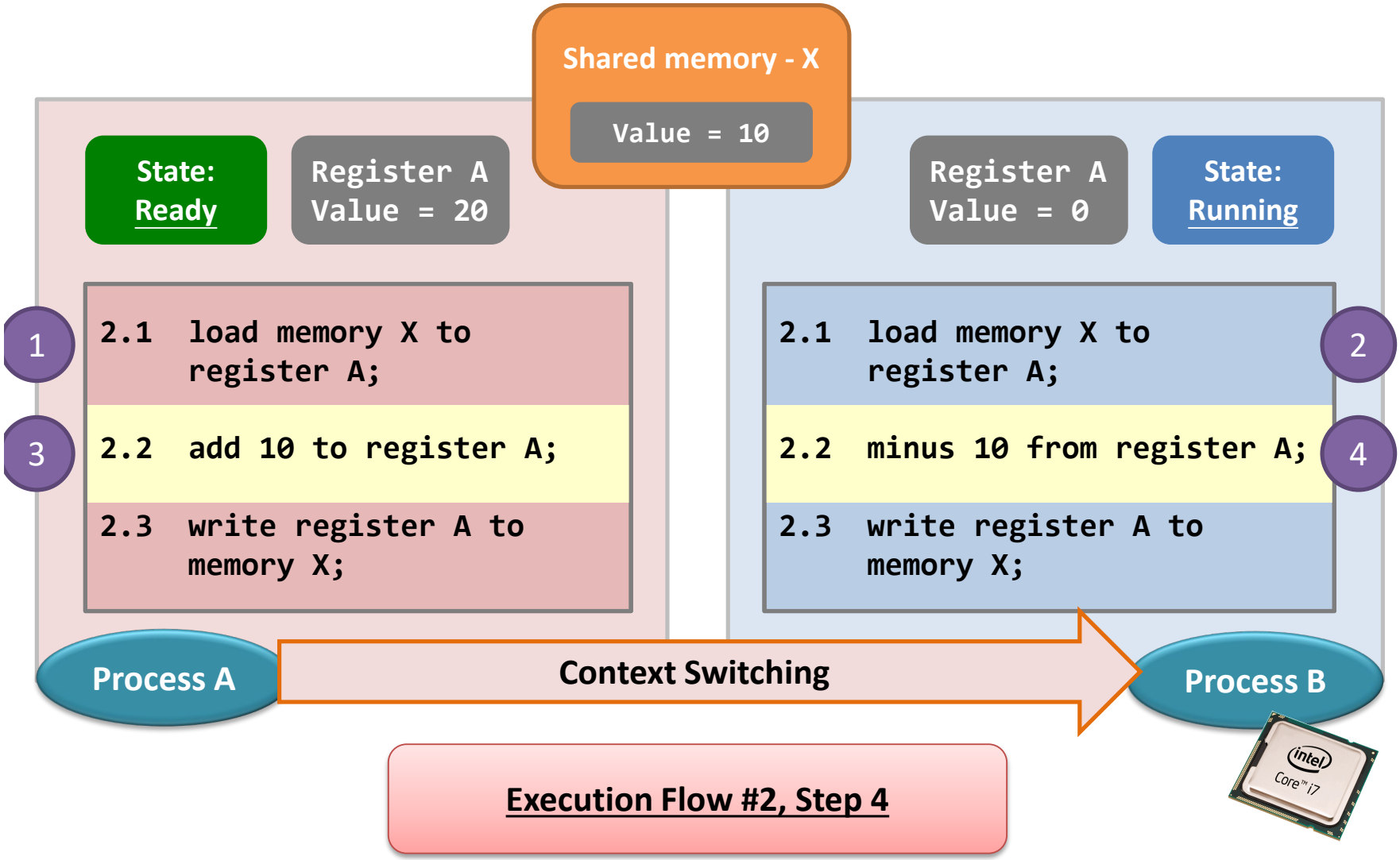
多进程共享内存问题


Shared memory - X
Value = 10
State: Ready
Register A Value = 10
Register A Value = 10
State: Running
1
2.1 load memory X to register A;
2.2 add 10 to register A;
2.3 write register A to memory X;
2.1 load memory X to register A;
2.2 minus 10 from register A;
2.3 write register A to memory X;
2
Process A
Context Switching
Process B
Execution Flow #2, Step 2

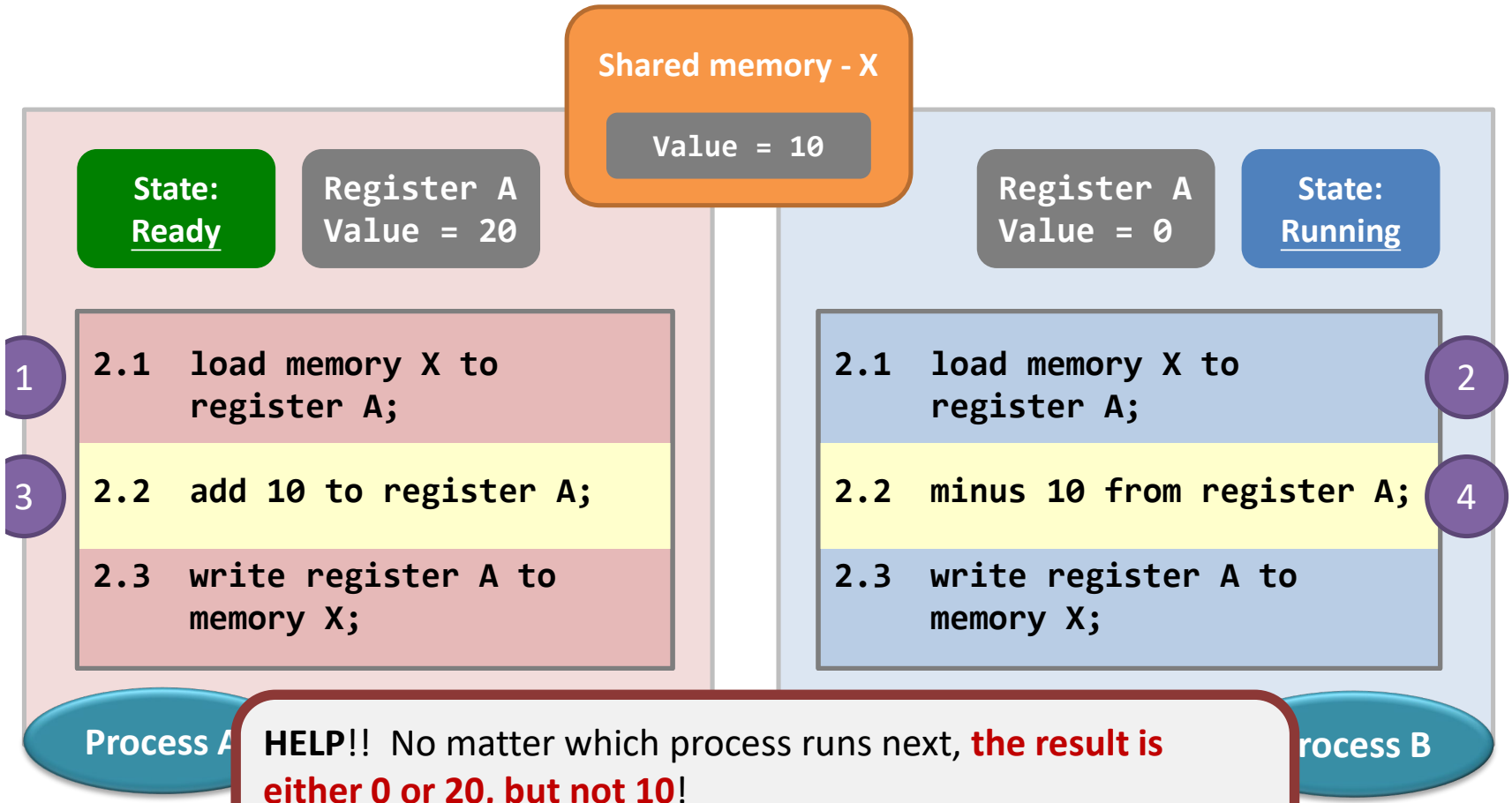
多进程共享内存问题


Shared memory - X
Value = 10
State: Running
Register A Value = 20
Register A Value = 10
State: Ready
1
2.1 load memory X to register A;
3
2.2 add 10 to register A;
2.3 write register A to memory X;
2
2.1 load memory X to register A;
2.2 minus 10 from register A;
2.3 write register A to memory X;
Process A
Context Switching
Process B
Execution Flow #2, Step 3

多进程共享内存问题

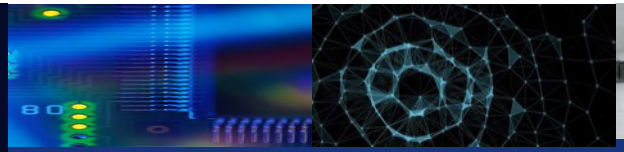

Shared memory - X
Value = 10
State: Ready
Register A Value = 20
Register A Value = 0
State: Running
1
2.1 load memory X to register A;
3
2.2 add 10 to register A;
2.3 write register A to memory X;
2.1 load memory X to register A;
2
2.2 minus 10 from register A;
4
2.3 write register A to memory X;
Process A
Context Switching
Process B
Execution Flow #2, Step 4

多进程共享内存问题



HELP!! No matter which process runs next, **the result is either 0 or 20, but not 10**!

The final result depends on the execution sequence!



竞争条件

- 上述两个例子均属于竞争条件（race condition）

- 竞争条件

—多个进程并发访问和操作同一数据并且执行结果与访问顺序有关；

—为了防止竞争条件，需要确保一次只有一个进程访问关键数据；

—竞争条件在操作系统中普遍存在；

—竞争条件是不确定的；

进程同步涉及到的话题

Cooperating Processes

concurrent accesses suffer from **race condition**

Solution

Process Sychronization

Guarantee **mutual exclusion**

Application

Semaphore Usage

Avoid **deadlock**

Idea: How to achieve

Define **critical section**

How to implement

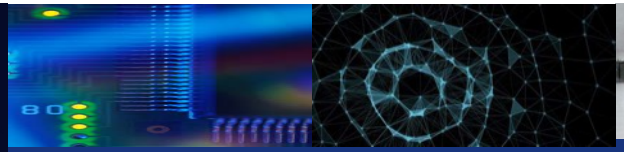
- Four requirements
- Software-based proposals
  - Disabling interrupts
  - strict alternation
  - peterson's solution
  - mutex lock
  - **Semaphore (best choice)**

Classic problems

- Producer-consumer problem
- Dining philosopher problem
- Reader-writer problem

与并发相关的关键术语

atomic operation	A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.



进程的交互

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	•Results of one process independent of the action of others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation •Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Deadlock (consumable resource) •Starvation

几个进程可能既表现出竞争，又表现出合作

临界区问题

- 考虑n个进程{P0，P1，…PN-1 }的系统
- 每个进程都有一段代码，称为临界区（Critical Section）：
  - 进程可能是更改公共变量、更新表、写入文件等。
  - 当一个进程处于临界区时，其他进程不得处于其临界区；
- 临界区问题通过设计协议来解决这个问题，能够协作进程。
- 每个进程必须在入口区（entry section）请求进入临界区的许可，临界区之后是退出区（exit section），然后是剩余区（remainder section）。

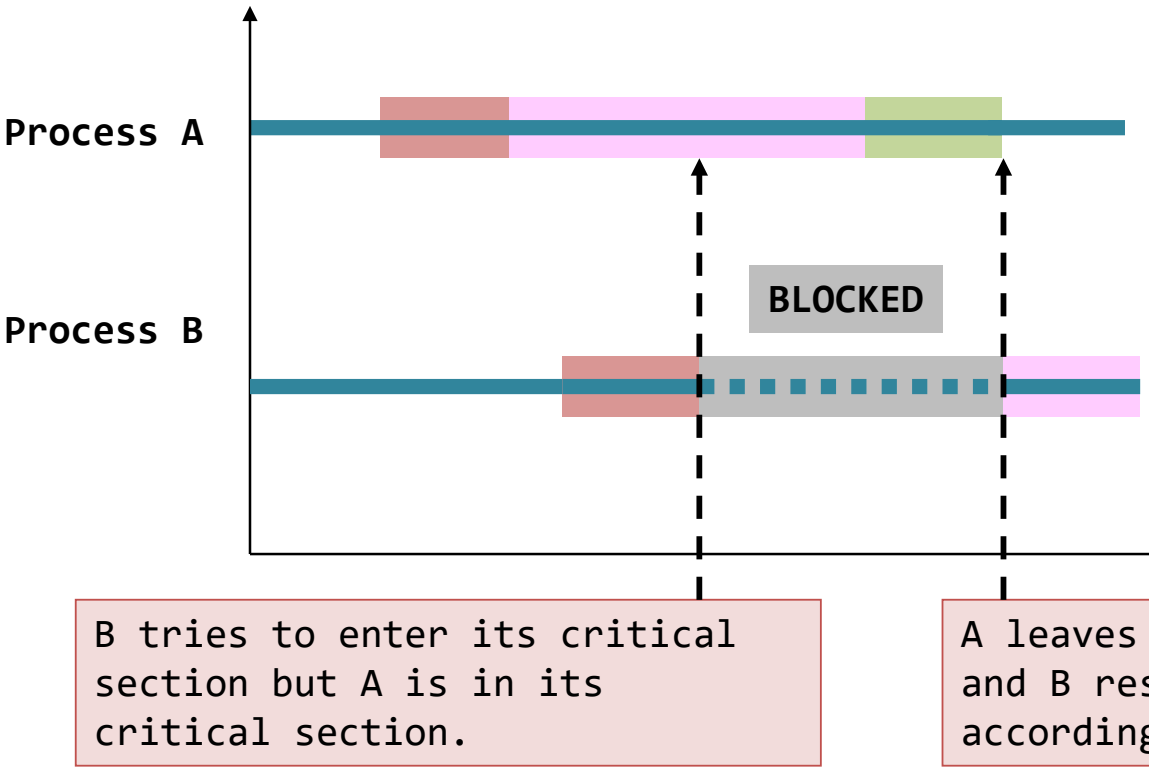
临界区问题

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

典型进程临界区的通用结构

进程在临界区中的活动过程

Remember, it is always the entry blocks other processes, but not the critical section.



Keys

- Critical section entry
- Inside Critical section
- Critical section exit
- Shared object (if any)

We will be using this coloring scheme throughout this part.

临界区问题解决访问的要求

1. 互斥（Mutual exclusion）-如果进程Pi在其临界区执行，则其他进程不能在其临界区执行；

2. 推进（progress）-如果没有进程在其临界区执行，并且存在一些希望进入其临界区的进程，则不能无限期推迟选择下一个将进入临界区的进程；

3. 有限等待（Bounded waiting）-在进程发出进入其临界区的请求后，以及在该请求被批准之前，其他进程允许进入其临界区的次数必须存在上限；
   - 假设每个进程以非零速度执行；
   - 关于n个进程的相对速度以及处理器的速度并没有任何假设；

操作系统中的竞争

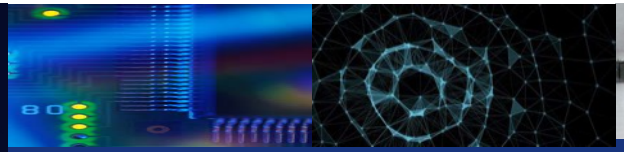
- 竞争的控制不可避免涉及操作系统；
- 进程自身需要以某种方式表达互斥的需求，在使用前加锁；
- 任何一种解决方案都需要操作系统的支持，提供锁机制；

```
        PROCESS 1 */

void P1
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

```
        /* PROCESS 2 */

void P2
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

. . .

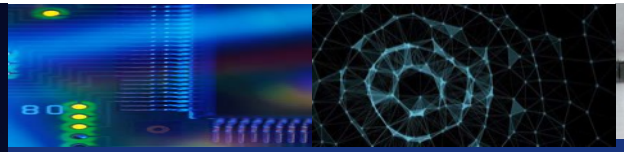
```
        /* PROCESS n */

void Pn
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```



操作系统

实现这些互斥条件的方法有多种：

- 其中一种方法是**软件方法**：即让并发执行的进程承担，需要与另一个进程合作，而不需要程序设计语言或者操作系统提供任何支持；
- 另一种方式涉及**专用机器指令**，这种方法的优点是减少开销，但是难以通用；
- 第三种方法是在**操作系统或程序设计语言**中提供某种级别的支持；

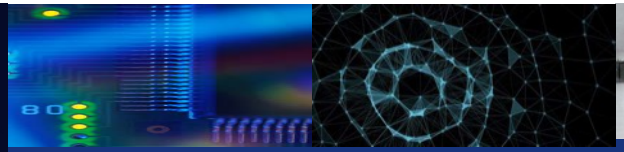


硬件同步

- 中断禁用
  - 单处理器系统；
  - 禁用中断保障互斥；

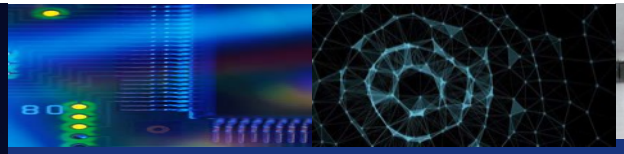
```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

- 缺点：
  - 代价非常高，执行的效率会明显下降；
  - 这种方法不能用于多处理器场景，在这种情况下，禁用中断不能保证互斥；



硬件同步

- 特殊的硬件指令，允许我们测试和修改一个字的内容，或以原子方式（不间断地）交换两个字的内容
  - 测试和设置指令 （test_and_set()）
  - 比较和交换指令（compare_and_swap()）



test_and_set()

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

定义

```
do {
   while (test_and_set(&lock))
      ; /* do nothing */

      /* critical section */

   lock = false;

      /* remainder section */
} while (true);
```

实现互斥

执行过程是原子的

compare_and_swap()

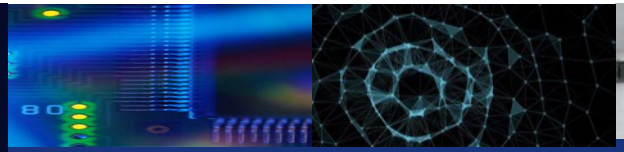
```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

Compare_and_set() 函数定义，执行是原子的

```
while (true) {
   while (compare_and_swap(&lock, 0, 1) != 0)
      ; /* do nothing */

      /* critical section */

   lock = 0;

      /* remainder section */
}
```

Compare_and_swap()的互斥实现

进入临界区，原始值等于所期待的值；退出临界区，它将lock设置回0



满足有限等待的test_and_set()

上述两种方法能够满足互斥的要求，但是还不能作为临界区问题的解决方案，为什么？

不满足有限等待的要求

```
boolean waiting[n];
boolean lock;
```

添加新的数据结构

```
do {
   waiting[i] = true;
   key = true;
   while (waiting[i] && key)
      key = test_and_set(&lock);
   waiting[i] = false;

      /* critical section */

   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;

   if (j == i)
      lock = false;
   else
      waiting[j] = false;

      /* remainder section */
} while (true);
```

特殊机器指令的优势：

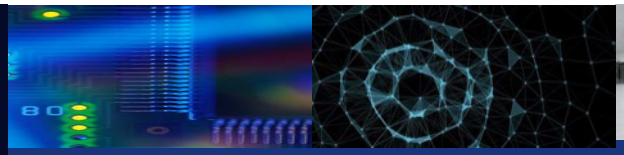
- Applicable to any number of processes on either a single processor or multiple processors sharing main memory;
- Simple and easy to verify;
- It can be used to support multiple critical sections; each critical section can be defined by its own variable;

特殊机器指令的劣势：

- Busy-waiting is employed, thus while a process is waiting for access to a critical section it continues to consume processor time;
- Starvation is possible when a process leaves a critical section and more than one process is waiting
- Deadlock is possible;

互斥锁

- 前面基于硬件的解决方案很复杂，程序员不能直接使用；
- 操作系统设计者构建软件工具来解决临界区问题；
- 最简单的是互斥锁（mutex）
  - 每一个互斥锁是表示锁是否可用的布尔变量；
- 通过以下方式保护临界区：
  - 首先获取一个锁acquire()；
  - 然后释放锁release()；
- 对acquire（）和release（）的调用必须是原子的
  - 通常通过硬件原子指令（如比较和交换）实现。



互斥锁

```
acquire() {
    while (!available)
       ; /* busy wait */
    available = false;
}
```

获得锁

```
release() {
   available = true;
}
```

释放锁

```
while (true) {
     acquire lock
          critical section
     release lock
          remainder section
}
```

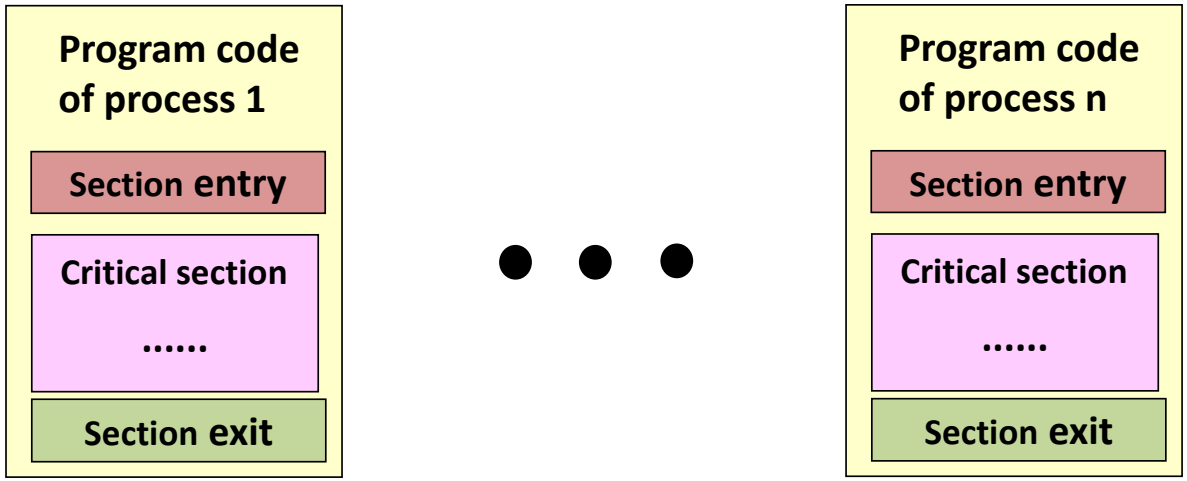
利用互斥锁的临界区问题解决方案

- 但是这个解决方案需要繁忙的等待（Busy Waiting）
  因此，该锁称为自旋锁（Spinlock）；
- 存在优点：当进程在等待锁时，没有上下文切换，在使用锁的时间较短时，自旋锁是有用的；自旋锁通常用于多处理器系统；

其他软件解决方案

—— 严格备选（Strict Alternation）

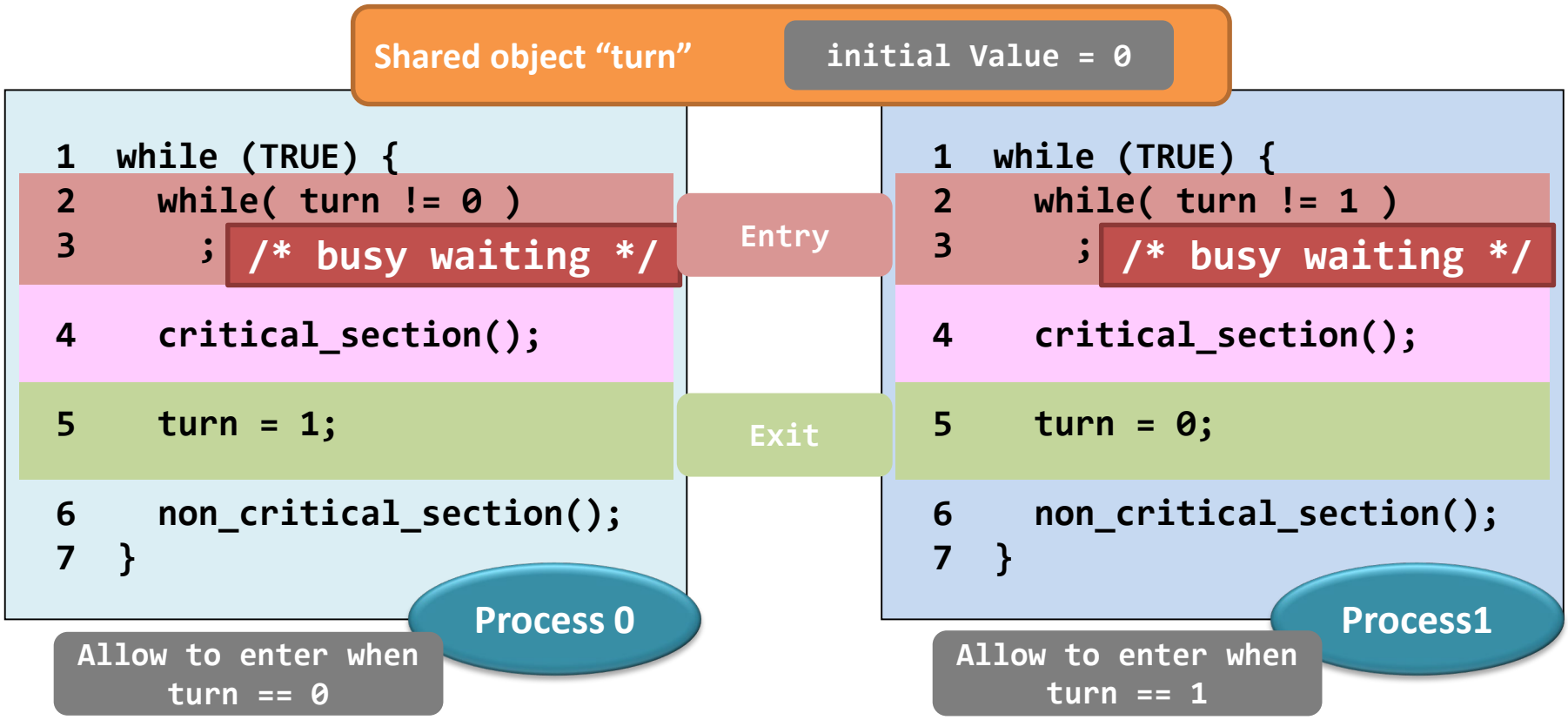
- Aim
  - To decide which process could go into its critical section



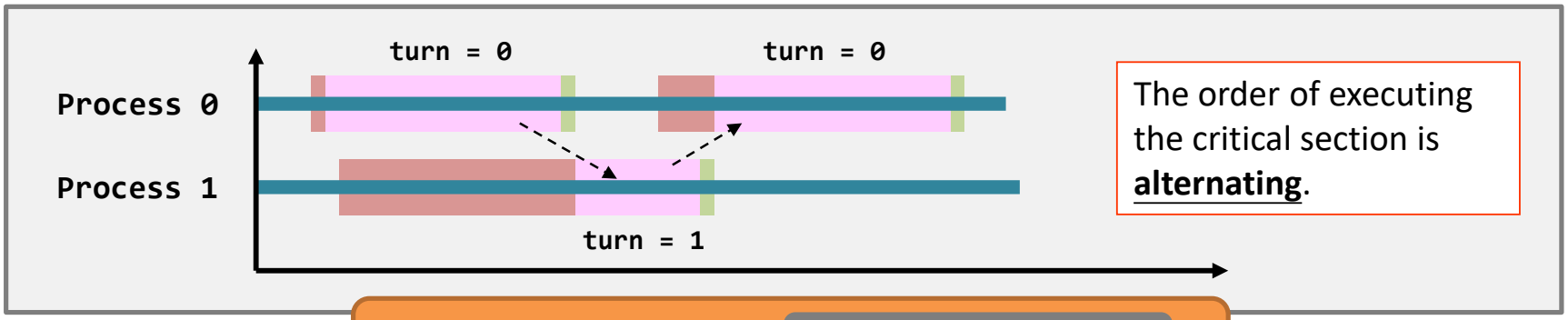
- Key Issues
  - Detect the status of processes (section entry)
    - Need other shared variables
  - Atomicity of section entry and exit

严格备选（Strict Alternation）

- **Method**
  - Using a new shared object to detect the status of other processes



严格备选（Strict Alternation）



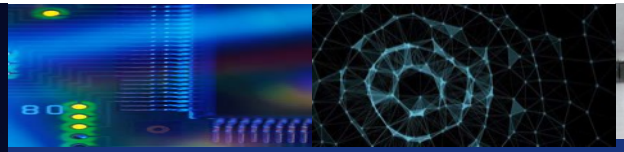
Shared object "turn"　　initial Value = 0

```
1  while (TRUE) {
2     while( turn != 0 )
3        ; /* busy waiting */

4     critical_section();

5     turn = 1;

6     non_critical_section();
7  }
```

Process 0

```
1  while (TRUE) {
2     while( turn != 1 )
3        ; /* busy waiting */

4     critical_section();

5     turn = 0;

6     non_critical_section();
7  }
```

Process1



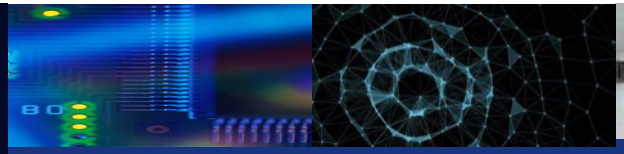
严格备选（Strict Alternation）

缺点：

- Strict alternation seems good, yet, it is **inefficient**.
  - Busy waiting wastes CPU resources.

- In addition, the alternating order is **too strict**.
  - What if Process 0 wants to enter the critical section **twice in a row**? **NO WAY!**
  - Violate any requirement?

Requirement #3. No process running outside its critical section should block other processes.



Peterson解决方案

- Peterson解决方案提供了解决临界区问题的一个很好的算法，并能说明满足互斥、进步、有限等待等要求的软件设计的复杂性。

- Highlights:
  - Share two data items
    - **`int turn;`** //whose turn to enter its critical section
    - **`Boolean interested[2];`** //if a process wants to enter
  - Processes would act as a gentleman: if you want to enter, I'll let you first
  - No alternation is there

Peterson解决方案

Shared object: “turn” & “interested[2]”

```
1   int turn;                          /* who can enter critical section */
2   int interested[2] = {FALSE,FALSE};  /* wants to enter critical section*/
3
4   void enter_region( int process ) {  /* process is 0 or 1 */
5     int other;                        /* number of the other process */
6     other = 1-process;                /* other is 1 or 0 */
7     interested[process] = TRUE;       /* want to enter critical section */
8     turn = other;
9     while ( turn == other &&
                interested[other] == TRUE )
10       ;    /* busy waiting */
11  }
12
13  void leave_region( int process ) {    /* process: who is leaving */
14    interested[process] = FALSE;    /* I just left critical region */
15  }
```

Entry

Exit

Peterson解决方案

```
 1  int turn;
 2  int interested[2] = {FALSE,FALSE};
 3
 4  void enter_region( int process ) {
 5    int other;
 6    other = 1-process;
 7    interested[process] = TRUE;
 8    turn = other;
 9    while ( turn == other &&
               interested[other] == TRUE )
10      ;    /* busy waiting */
11  }
12
13  void leave_region( int process ) {
14    interested[process] = FALSE;
15  }
```

Line 8 therefore makes the other one the turn to run.

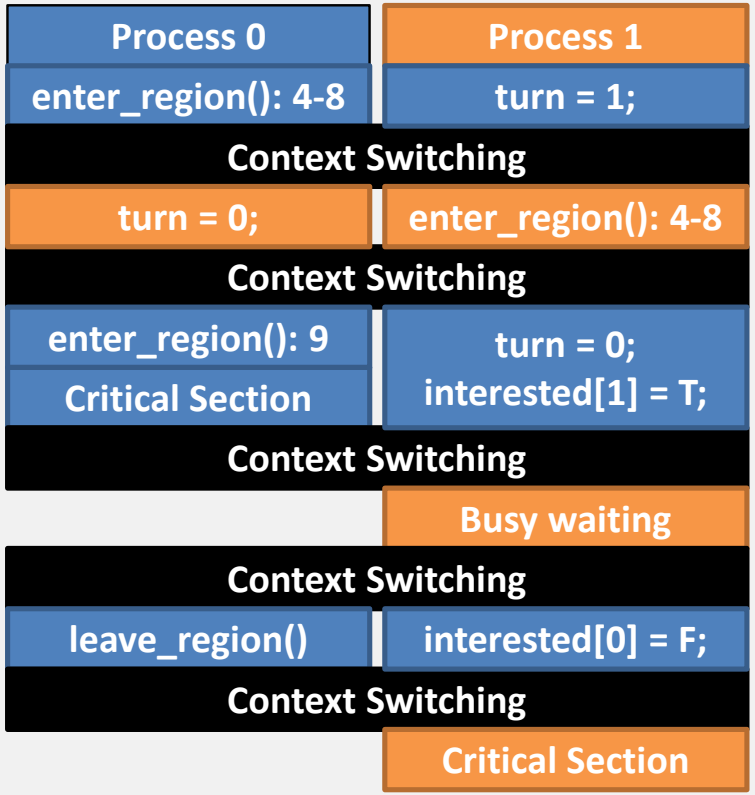
Of course, the process is willing to wait when she wants to enter the critical section.

"I'm a gentleman!"

The process always let another process to enter the critical region first although she wants to enter too.

Peterson解决方案

```
1  int turn;
2  int interested[2] = {FALSE,FALSE};
3
4  void enter_region( int process ) {
5    int other;
6    other = 1-process;
7    interested[process] = TRUE;
8    turn = other;
9    while ( turn == other &&
               interested[other] == TRUE )
10     ;    /* busy waiting */
11 }
12
13 void leave_region( int process ) {
14   interested[process] = FALSE;
15 }
```

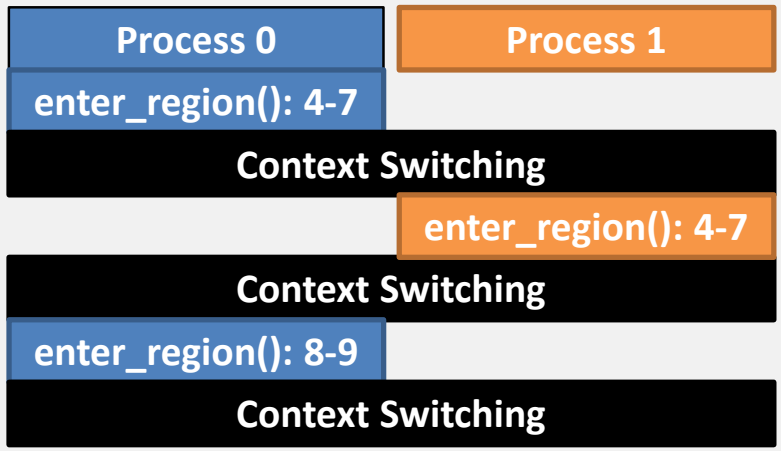


and the story goes on…

Can you show that the requirements are satisfied?

Peterson解决方案

```
 1  int turn;
 2  int interested[2] = {FALSE,FALSE};
 3
 4  void enter_region( int process ) {
 5    int other;
 6    other = 1-process;
 7    interested[process] = TRUE;
 8    turn = other;
 9    while ( turn == other &&
               interested[other] == TRUE )
10      ;    /* busy waiting */
11  }
12
13  void leave_region( int process ) {
14    interested[process] = FALSE;
15  }
```

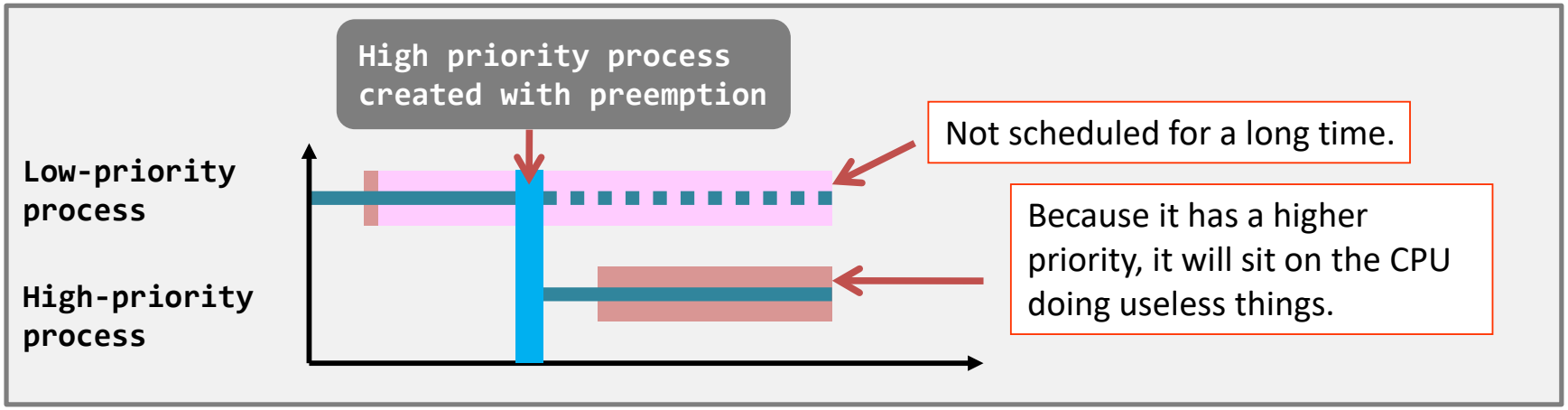


Can you complete the flow?
(what is the difference?)

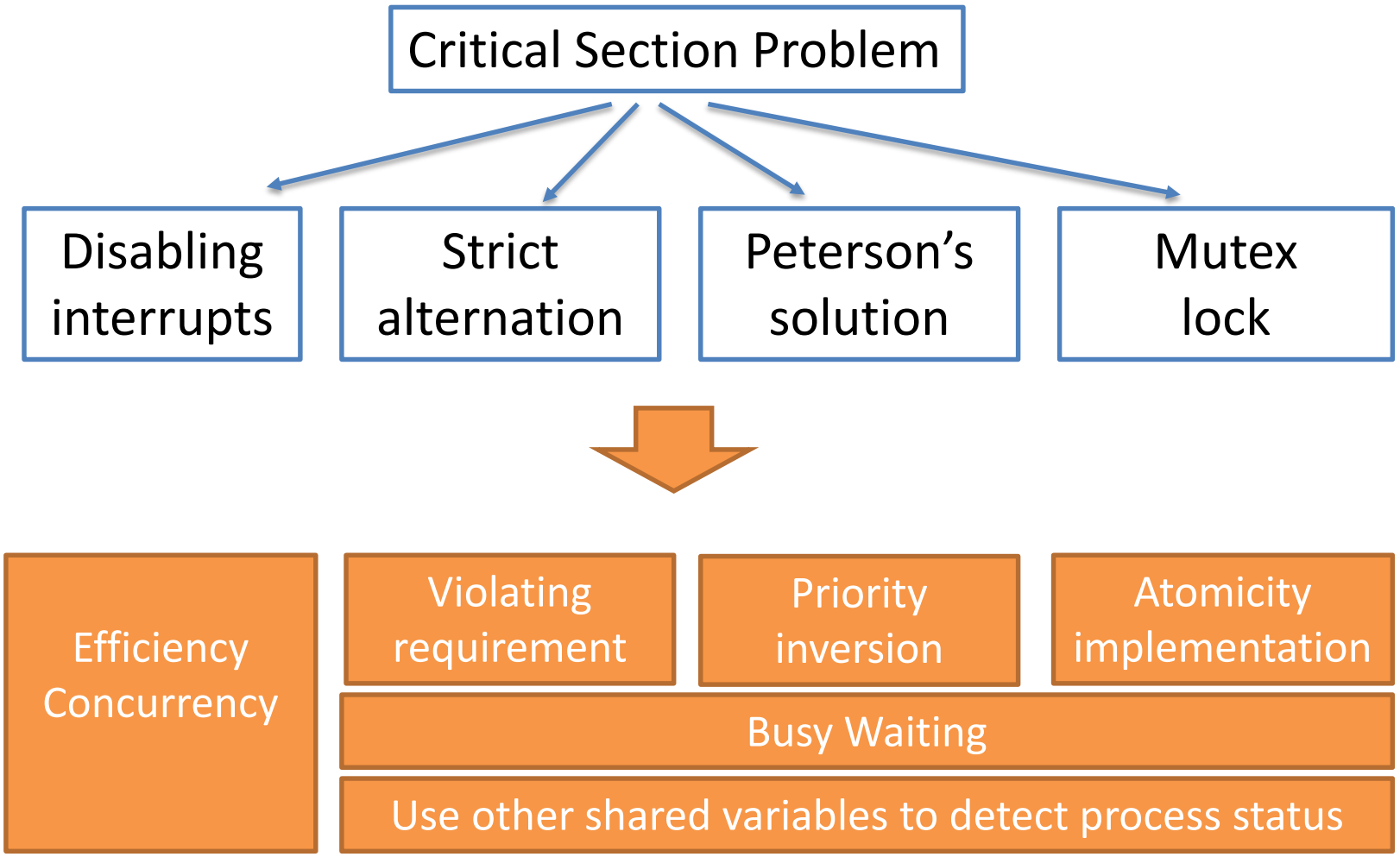
Can both processes progress?

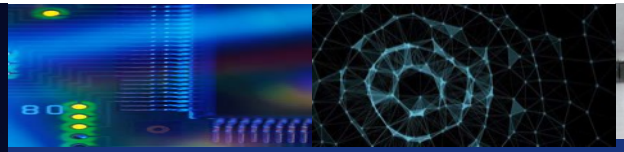
Peterson解决方案

- Busy waiting has its own problem…
  - **An apparent problem**: wasting CPU time.
  - **A hidden, serious problem**: **priority inversion problem.**
    - A low priority process is inside the critical region, but …
    - A high priority process wants to enter the critical region.
    - Then, the high priority process will perform busy waiting for a long time or even forever.



至此主要的解决方案


Critical Section Problem
Disabling interrupts
Strict alternation
Peterson's solution
Mutex lock
Efficiency
Concurrency
Violating requirement
Priority inversion
Atomicity implementation
Busy Waiting
Use other shared variables to detect process status




信号量

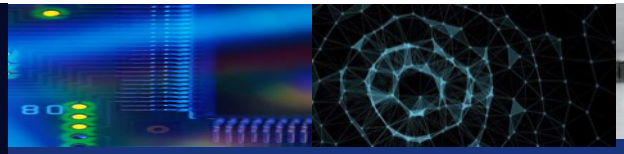
- 互斥锁是一种外部强加的控制方案，或者说是全局控制；
- 信号量是一种内部主动的控制方案，或者说是局部控制；

- 同步工具，为进程同步其活动提供更复杂的方法（比互斥锁）。
- 信号量S－整数变量
- 只能通过两个不可分割（原子）操作访问，如何实现？
  - Wait（）和Signal（）
    - 最初称为P（）和V（）
- wait（）操作的定义

```
Wait(S){
While（S<=0）
; // busy wait
  S--;
}
```

- signal（）操作的定义

```
Signal(S){
  S++;
}
```

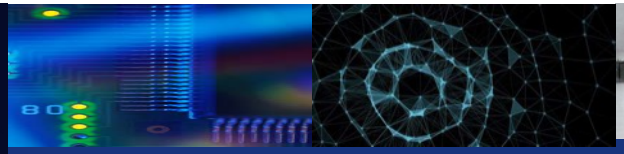


信号量—重要结论

There is no way to know before a process decrements a semaphore whether it will block or not

There is no way to know which process will continue immediately on a uniprocessor system when two processes are running concurrently

You don't know whether another process is waiting so the number of unblocked processes may be zero or one



计数信号量定义

计数信号量：整数值可以在不受限制的域内变化

```
struct semaphore {
      int count;
      queueType queue;
};
void semWait(semaphore s)
{
      s.count--;
      if (s.count < 0) {
            /* place this process in s.queue */;
            /* block this process */;
      }
}
void semSignal(semaphore s)
{
      s.count++;
      if (s.count <= 0) {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
      }
}
```

二元信号量定义

```
struct binary_semaphore {
     enum {zero, one} value;
     queueType queue;
};
void semWaitB(binary_semaphore s)
{
     if (s.value == one)
          s.value = zero;
     else {
            /* place this process in s.queue */;
            /* block this process */;
     }
}
void semSignalB(semaphore s)
{
     if (s.queue is empty())
          s.value = one;
     else {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
     }
}
```

区分二元信号量和互斥锁（mutex）：整数值只能介于0和1之间；

二元信号量与互斥锁的不同：加锁和解锁的主体不同！

强弱信号量

* **A queue is used to hold processes waiting on the semaphore**

强信号量

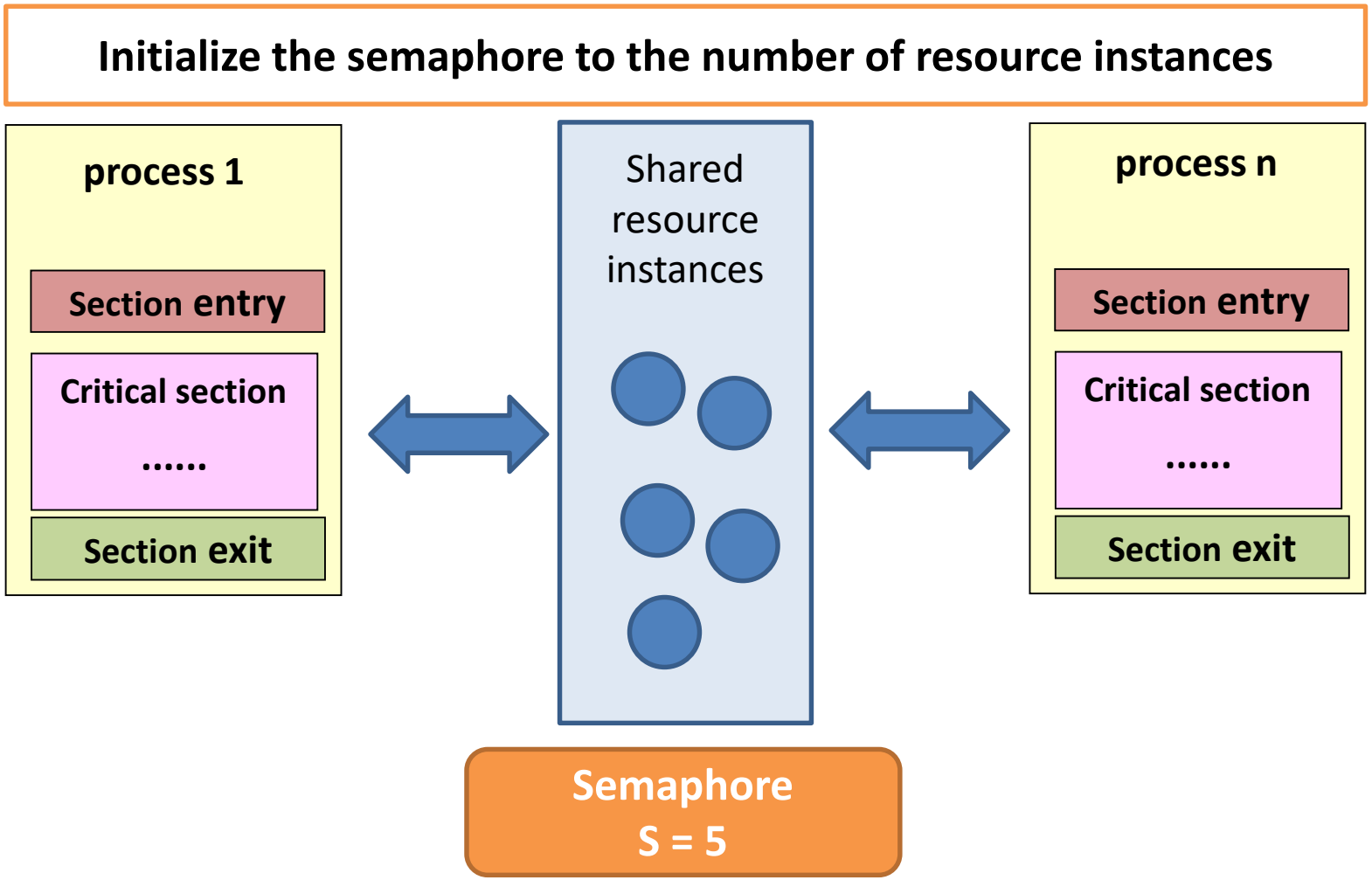
- 采用FIFO，被阻塞时间最长的进程最先从队列释放；

弱信号量

- 没有规定从队列中移出顺序的信号量；

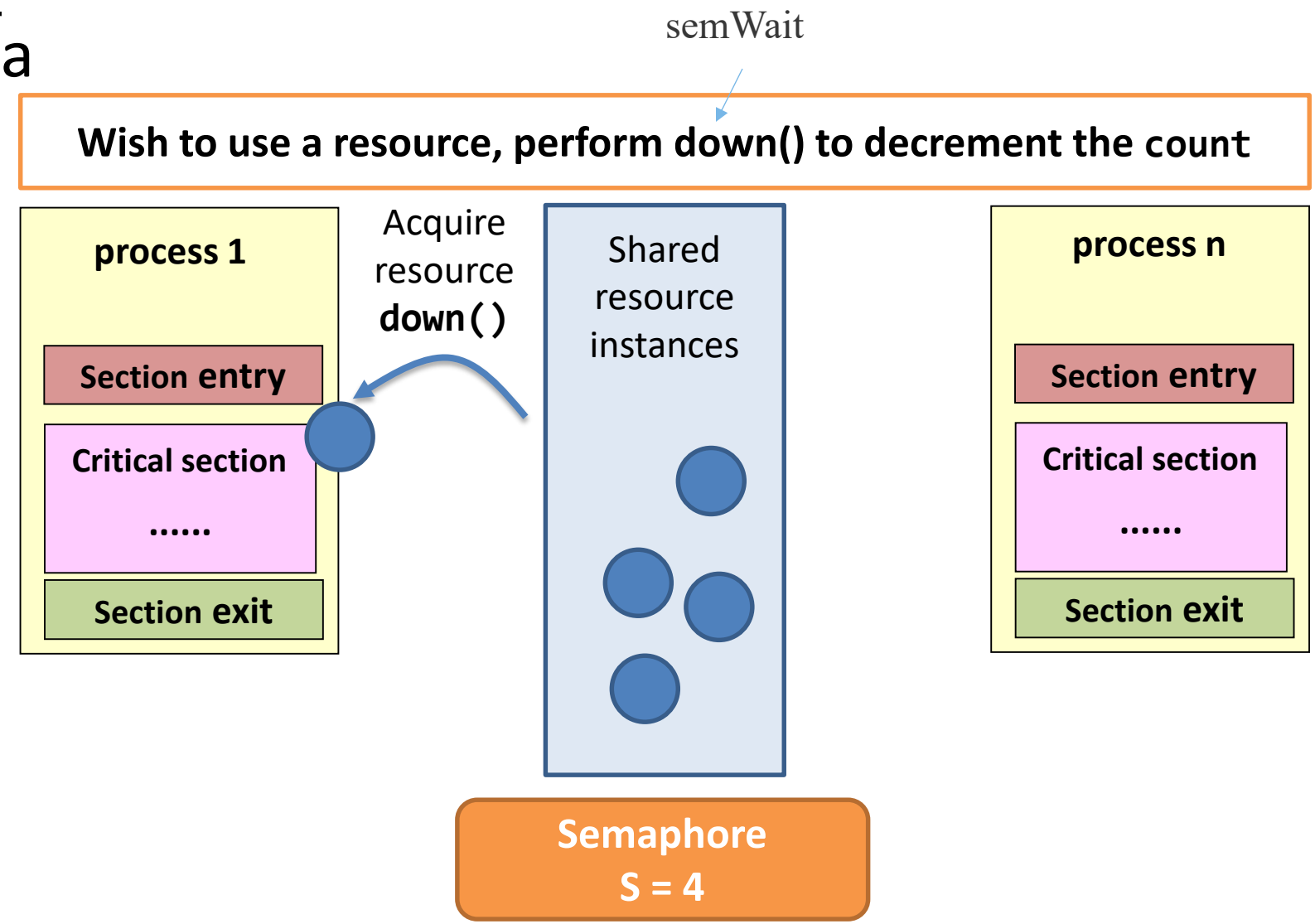
信号量

- Idea


Initialize the semaphore to the number of resource instances
process 1
Section entry
Critical section
.....
Section exit
Shared resource instances
process n
Section entry
Critical section
.....
Section exit
Semaphore
S = 5

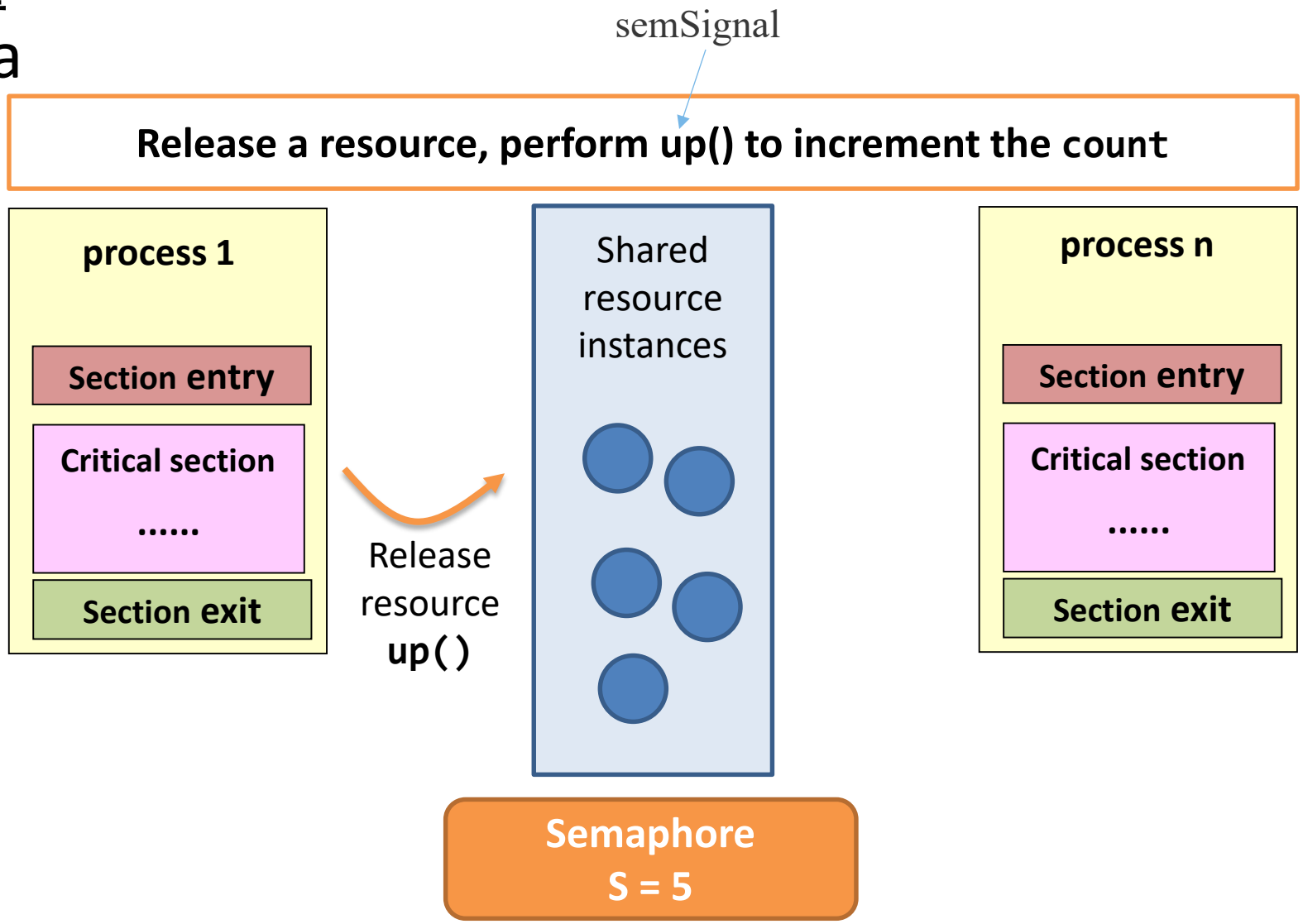
信号量

- Idea


semWait
Wish to use a resource, perform down() to decrement the count
process 1
Section entry
Critical section
.....
Section exit
Acquire resource down()
Shared resource instances
process n
Section entry
Critical section
.....
Section exit
Semaphore
S = 4

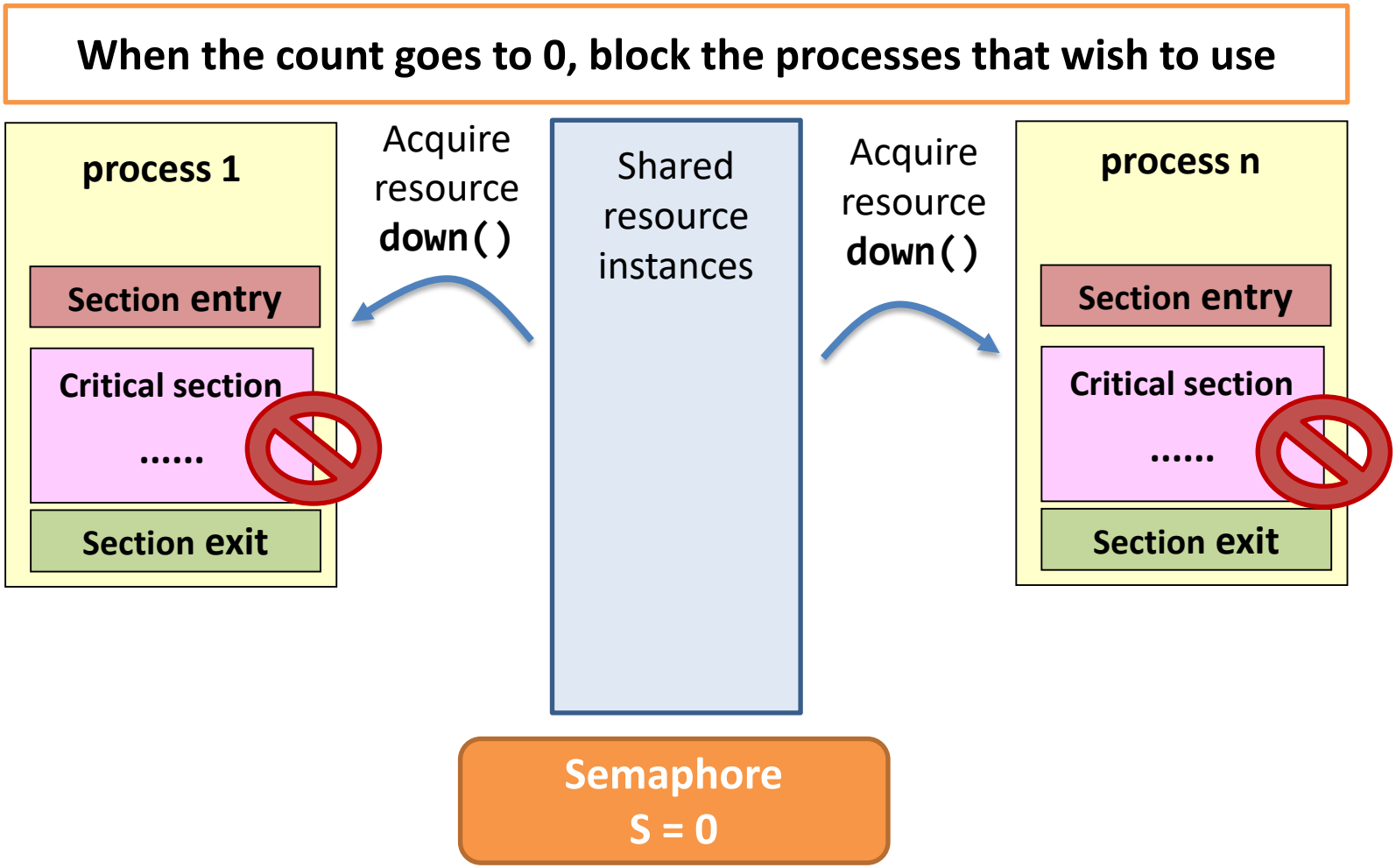
信号量

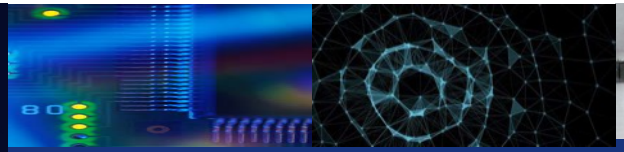
- Idea


semSignal
Release a resource, perform up() to increment the count
process 1
Section entry
Critical section
.....
Section exit
Release resource up()
Shared resource instances
process n
Section entry
Critical section
.....
Section exit
Semaphore
S = 5

信号量

- Idea


When the count goes to 0, block the processes that wish to use
process 1
Section entry
Critical section
.....
Section exit
Acquire resource down()
Shared resource instances
Acquire resource down()
process n
Section entry
Critical section
.....
Section exit
Semaphore
S = 0



信号量——简单实现

Data Type definition

```
typedef int semaphore;
```

Counting Semaphore: initialized to be the number of resources available

Section Entry: down()

```
1  void down(semaphore *s) {
2
3     while ( *s == 0 ) {
4
5          ;//busy wait
6
7      }
8     *s = *s - 1;
9
10 }
```

Section Exit: up()

```
1  void up(semaphore *s) {
2
3
4
5     *s = *s + 1;
6
7  }
```

信号量——解决忙碌等待

Data Type definition

```
typedef int semaphore;
```

First issue: Busy waiting

Solution: block the process instead of busy waiting (place the process into a waiting queue)

Section Entry: down()

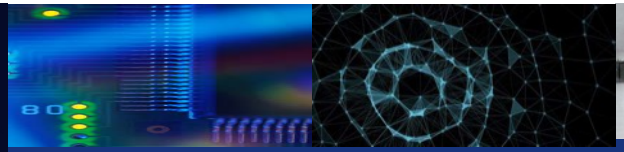
```
1  void down(semaphore *s) {
2
3     while ( *s == 0 ) {
4
5        special_sleep();
6
7      }
8     *s = *s - 1;
9
10 }
```

Block()

Section Exit: up()

```
1  void up(semaphore *s) {
2
3    if ( *s == 0 )
4      special_wakeup();
5    *s = *s + 1;
6
7  }
```

Wakeup ()



信号量——解决忙碌等待

Data Type definition

```
typedef int semaphore;
```

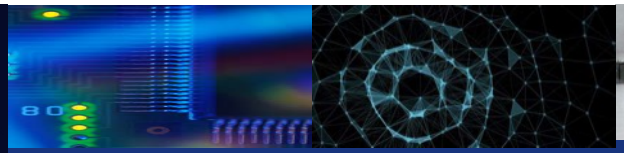
First issue: Busy waiting

Solution: block the process instead of busy waiting (place the process into a waiting queue)

```
typedef struct{

     int value;
     struct process * list;

}semaphore;
```

Note

Implementation: The waiting queue may be associated with the semaphore, so a semaphore is not just an integer



信号量——代码

Data Type definition

```
typedef int semaphore;
```

多处理器环境中，这种解决方案是否有效?

Section Entry: down()

```
1  void down(semaphore *s) {
2     disable_interrupt();
3     while ( *s == 0 ) {
4        enable_interrupt();
5        special_sleep();
6        disable_interrupt();
7      }
8     *s = *s - 1;
9     enable_interrupt();
10 }
```

Why need these two statements?

Disabling interrupts may sacrifice concurrency, so it is essential to keep the critical section as short as possible

Section Exit: up()

```
1  void up(semaphore *s) {
2    disable_interrupt();
3    if ( *s == 0 )
4      special_wakeup();
5    *s = *s + 1;
6    enable_interrupt();
7  }
```

信号量两种可能的实现方法

```
semWait(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
        /* do nothing */;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue*/;
        /* block this process (must also set s.flag to 0)
*/;
    }
    s.flag = 0;
}

semSignal(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
         /* do nothing */;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    s.flag = 0;
}
```

(a) Compare and Swap Instruction

```
semWait(s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process and allow interrupts */;
    }
    else
       allow interrupts;
}

semSignal(s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    allow interrupts;
}
```

(b) Interrupts

信号量——细节

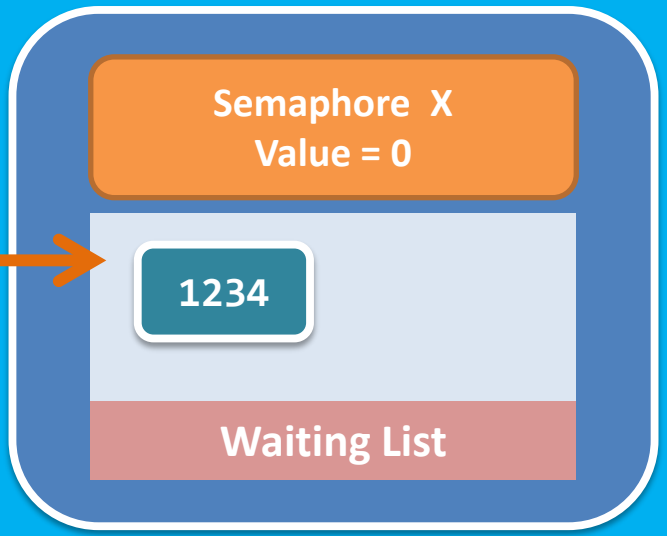
Process 1234

down(X)

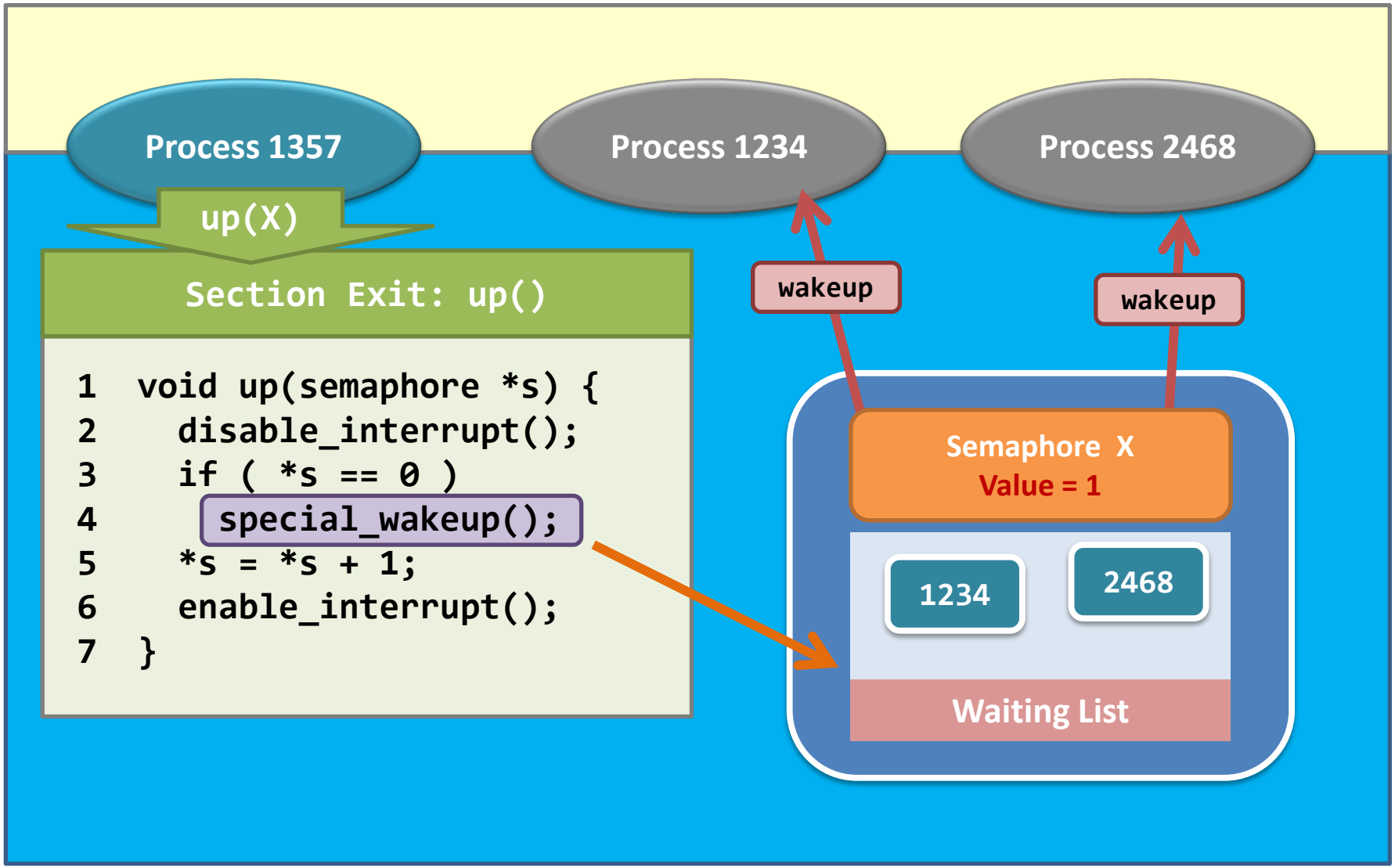
Suppose that process 1234 is willing to access the shared resource (enter its critical section), but no resource is available

Section Entry: down()

```
1  void down(semaphore *s) {
2     disable_interrupt();
3     while ( *s == 0 ) {
4        enable_interrupt();
5        special_sleep();
6        disable_interrupt();
7      }
8     *s = *s - 1;
9     enable_interrupt();
10 }
```



信号量——细节


Process 1357
Process 1234
Process 2468
up(X)
Section Exit: up()
1 void up(semaphore *s) {
2 disable_interrupt();
3 if (*s == 0)
4 special_wakeup();
5 *s = *s + 1;
6 enable_interrupt();
7 }
wakeup
wakeup
Semaphore X
Value = 1
1234
2468
Waiting List

信号量——细节

Process 1234

Process 2468

down(X)

down(X)

Note that it is impossible for **two blocked processes to get out of the down( ) simultaneously.**

Why?

Only one process can invoke **disable_interrupt()**

Only one process can manipulate this shared variable

Section Entry: down()

```
 1  void down(semaphore *s) {
 2     disable_interrupt();
 3     while ( *s == 0 ) {
 4        enable_interrupt();
 5        special_sleep();
 6        disable_interrupt();
 7     }
 8     *s = *s - 1;
 9     enable_interrupt();
10  }
```

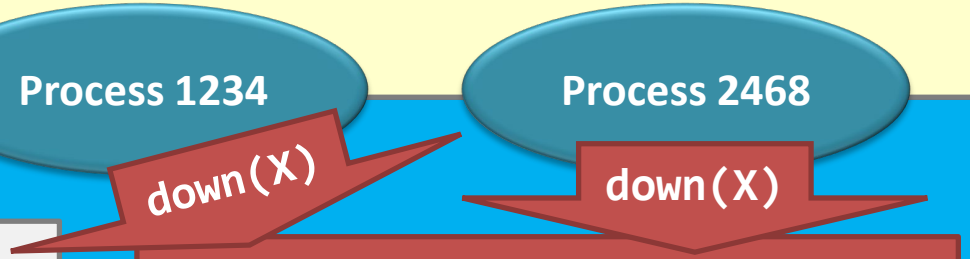
here

信号量——细节

Note that it is impossible for **two processes to get out of the down() simultaneously.**

Why?

Whether which process can get out of **down()** is **the business of the scheduler**.



Section Entry: down()

```
 1  void down(semaphore *s) {
 2     disable_interrupt();
 3     while ( *s == 0 ) {
 4        enable_interrupt();
 5        special_sleep();
 6        disable_interrupt();
 7      }
 8     *s = *s - 1;
 9     enable_interrupt();
10  }
```

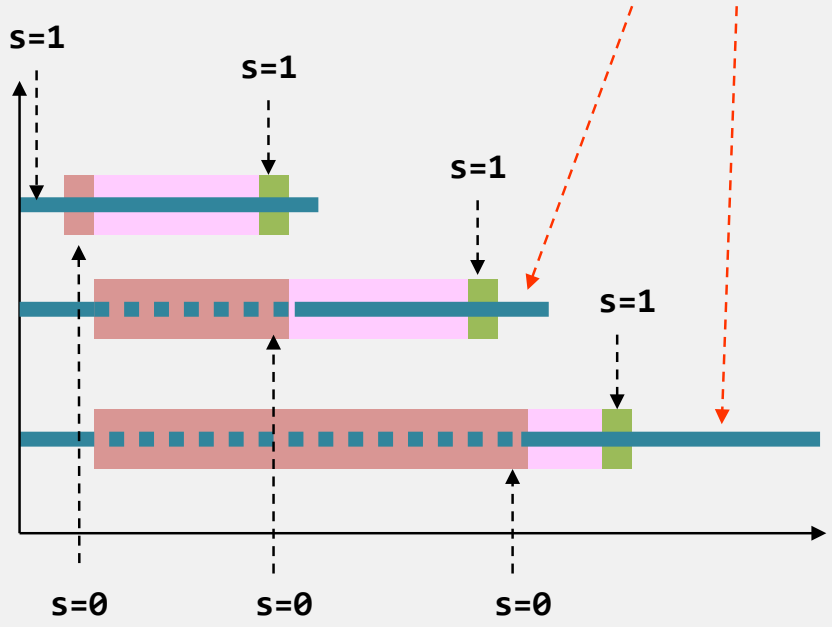
here

信号量——总体行为

- Add them together…

是否完全解决了忙等待的问题?

Either one of the processes can enter the critical section when the first process calls "**up(s)**".

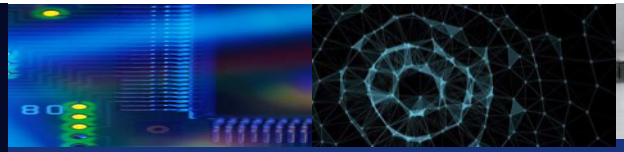



```
semaphore *s;
*s = 1;        /* initial value */
```

```
1  while(TRUE) {
2      down(s);
3      critical_section();
4      up(s);
5  }
```

entry

exit

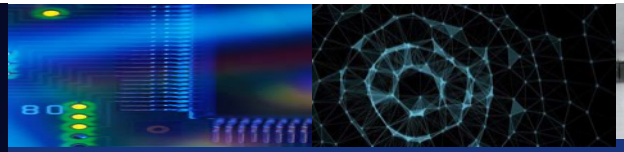


信号量的实现——Linux

内核为每个信号量集设置了一个semid_ds结构，定义在 <sys/sem.h>中：

```
1 struct semid_ds {
2     struct ipc_perm sem_perm;  /* Ownership and permissions */
3     time_t          sem_otime; /* Last semop time */
4     time_t          sem_ctime; /* Last change time */
5     unsigned short  sem_nsems; /* No. of semaphores in set */
6 };
```

```
1 struct {
2     unsigned short semval;  /* 信号量值 */
3     pid_t          sempid;  /* 上一个操作进程 */
4     unsigned short semncnt; /* # 等待获取资源的进程数 */
5     unsigned short semznt;  /* # processes awaiting semval==0 */
6 };
```



信号量的实现——Linux

要获得一个信号量ID，要调用的第一个函数是semget；

```
1 #include <sys/sem.h>
2 int semget(key_t key, int nsems, int semflg);
3 返回值：若成功则返回信号量ID，出错则返回-1
```

semctl函数包含了多种信号量操作；

```
1 #include <sys/sem.h>
2 int semctl(int semid, int semnum, int cmd, ...);
3 返回值：成功返回一个整数，出错返回-1
```

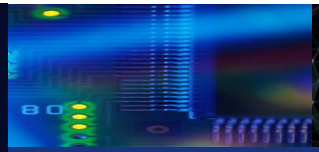
信号量的实现——Linux

函数semop用以操作一个信号量集；

```
#include <sys/sem.h>
int semop(int semid, struct sembuf *sops, unsigned nsops);
返回值：成功返回0，出错返回-1
```

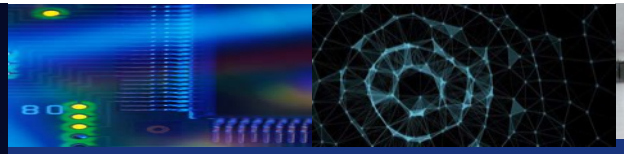
函数semop用以操作一个信号量集；

```
struct sembuf{
    unsigned short sem_num;    /* 要操作的信号集中的信号序号（默认0开始） */
    short          sem_op;     /* 对信号集的操作 */
    short          sem_flg;    /* 两个参数 IPC_NOWAIT, SEM_UNDO */
};
```



信号量的实现——Linux

```c
#include <sys/sem.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/ipc.h>

union semun {
    int              val;    /* Value for SETVAL */
    struct semid_ds *buf;    /* Buffer for IPC_STAT, IPC_SET */
    unsigned short  *array;  /* Array for GETALL, SETALL */
    struct seminfo  *__buf;  /* Buffer for IPC_INFO (Linux-specific) */
};

main()
{
    key_t       key;
    int         semid;
    int         r;
    union semun sems;

    key = ftok(".", 255);                   //得到key
    if (key == -1)
        perror("ftok error"), exit(-1);
    semid = semget(key, 1, IPC_CREAT|0666);     //创建一个只有一个信号量的信号集
    if (semid == -1)
        perror("semget error"), exit(-1);
    sems.val = 10;                              //初始信号值为10
    r = semctl(semid, 0, SETVAL, sems.val);     //将信号集中第0个元素设置为10
    if (r == -1)
        perror("semctl error"), exit(-1);

    while(1);

}
```



管程

- 虽然信号量提供了一种方便且有效的进程同步机制，但是它们的错误使用可能导致难以检测的时序错误，这些错误只在特定执行顺序时才会出现；

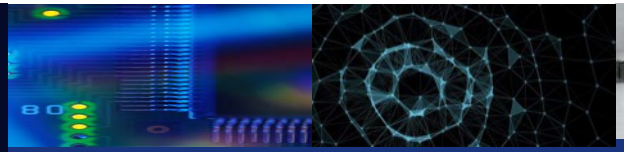
- 即使采用信号量，也不能完全解决时序问题；使用信号量产生的各种异常问题：

```
signal(mutex);
    ...
  critical section
    ...
wait(mutex);
```

交换Signal和Wait，多个进程在临界区中

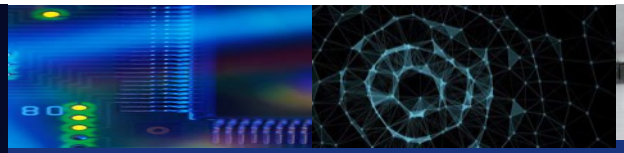
```
wait(mutex);
    ...
  critical section
    ...
wait(mutex);
```

利用Wait替换Signal，出现死锁



管程

- 管程是一种高级程序设计语言结构，它提供的功能与信号量相同，但更易于控制；
- 管程结构在很多程序设计语言中得以实现；
  - 包括并发Pascal，Pascal，Pascal-Plus，Modula-2，Modula-3，and Java
- 现在已经被作为一个程序库实现；
- 管程是由一个或多个进程、一个初始化序列和局部数据组成的软件模块；(使用信号的管程)

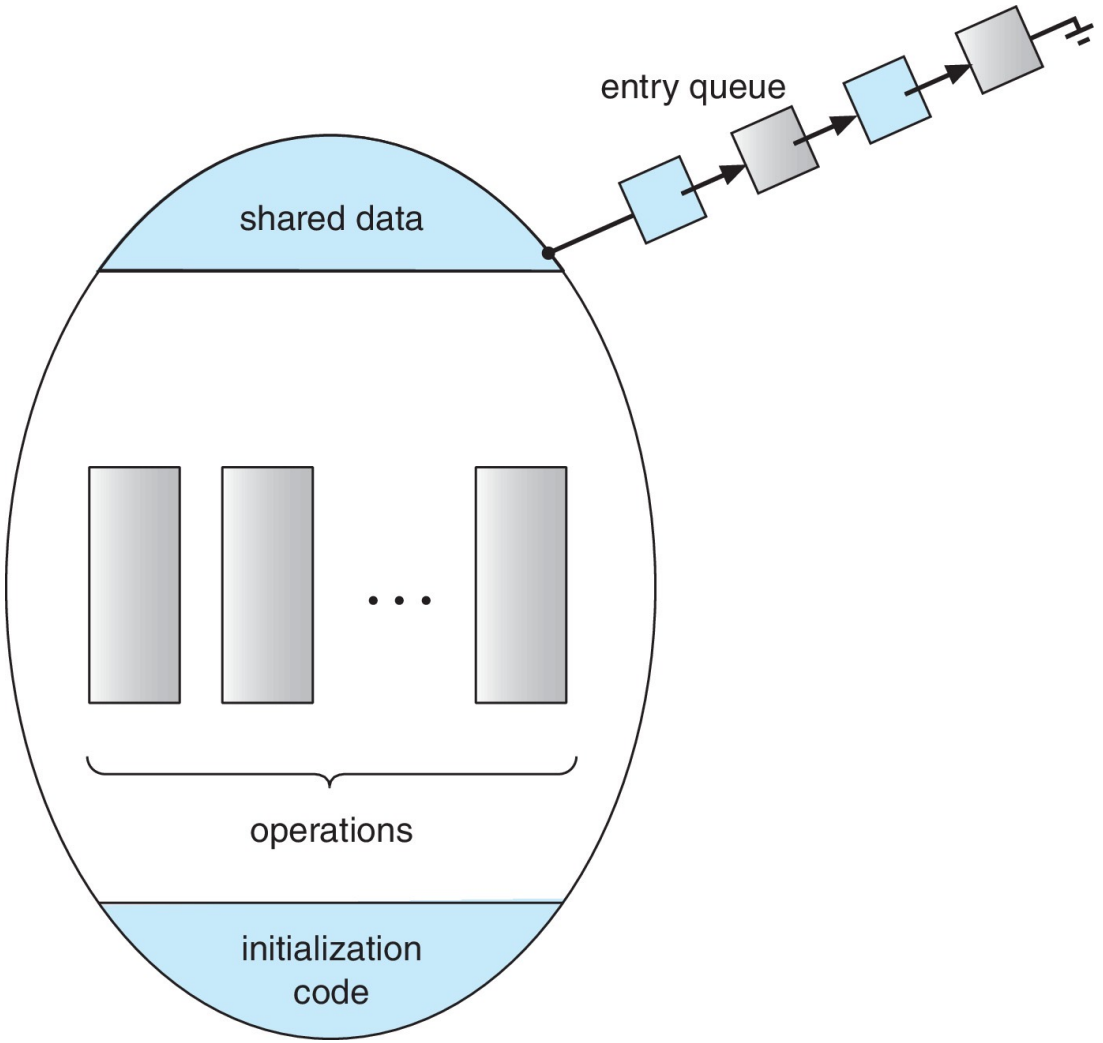


管程

- 管程类型属于抽象数据类型，封装了数据及对其操作的一组函数，内部变量只能由过程中的代码访问；
- 提供一组由程序员定义的在管程内互斥的操作；
- 管程中一次只能有一个进程处于活跃状态；
- 管程的结构定义如下：

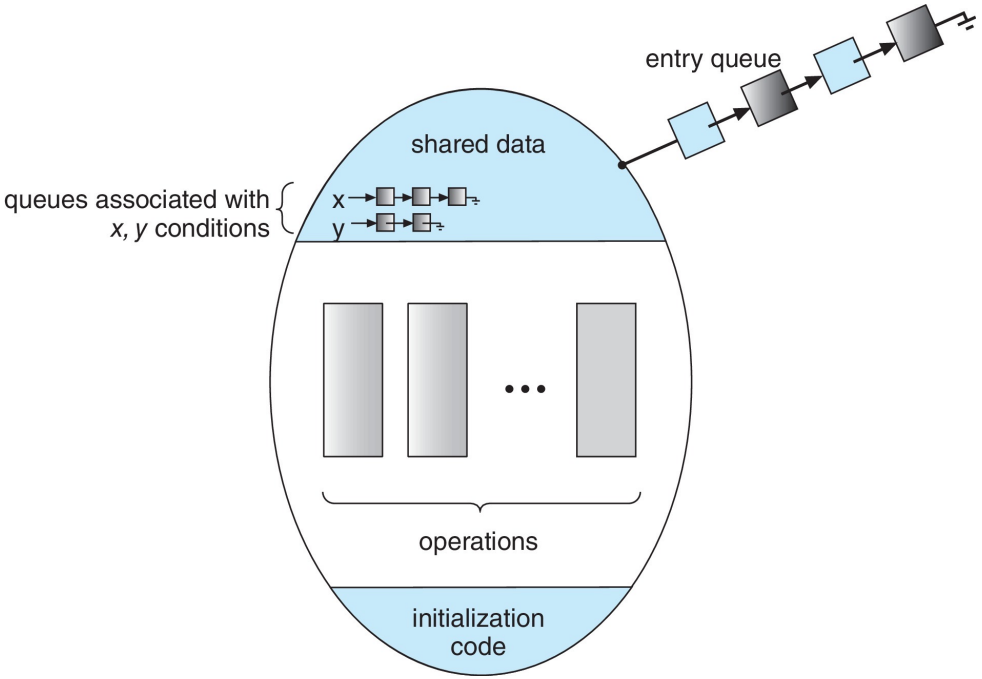
```
monitor monitor name
{
   /* shared variable declarations */

   function P1 ( . . . ) {
      . . .
   }

   function P2 ( . . . ) {
      . . .
   }

         .
         .
         .
   function Pn ( . . . ) {
      . . .
   }

   initialization_code ( . . . ) {
      . . .
   }
}
```

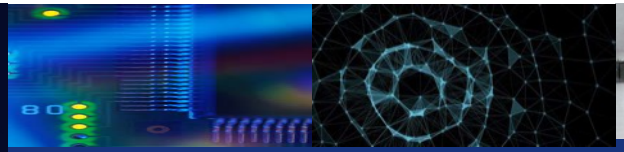
管程



条件变量

- 定义附加的同步机制，可以有条件（condition）结构来提供；
- 当程序员需要编写定制的同步方案时，可以定义一个或者多个类型为condition的变量；
- 对于条件变量，只有操作wait()和signal可以调用；



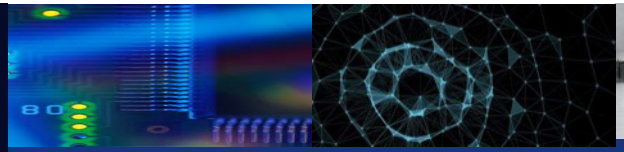


管程的特点

Local data variables are accessible only by the monitor's procedures and not by any external procedure

Process enters monitor by invoking one of its procedures

Only one process may be executing in the monitor at a time



条件变量的使用

- **Consider P_1 and P_2 that that need to execute two statements S_1 and S_2 and the requirement that S_1 to happen before S_2**
  - Create a monitor with two procedures F_1 and F_2 that are invoked by P_1 and P_2 respectively
  - One condition variable "x" initialized to 0
  - One Boolean variable "done"
  - **F1:**

```
S1;
done = true;
x.signal();
```

  - **F2:**

```
if done = false
   x.wait()
S2;
```

使用信号量实现管程

- Variables

```
semaphore mutex;  // (initially = 1)
semaphore next;   // (initially = 0)
int next_count = 0; // number of processes waiting
                    inside the monitor
```

- Each function *P* will be replaced by

```
wait(mutex);
    …
    body of P;
    …
if (next_count > 0)
    signal(next)
else
    signal(mutex);
```

- Mutual exclusion （互斥）within a monitor is ensured

使用条件变量实现管程

- For each condition variable *x*, we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation x.wait() can be implemented as:

```
x_count++;
if (next_count > 0)
      signal(next);
else
     signal(mutex);
wait(x_sem);
 x_count--;
```

使用条件变量实现管程

- The operation x.signal() can be implemented as:

```
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```

管程中的进程恢复

假设进程P 调用x.signal()，在条件变量上有一个等待进程Q；

- 唤醒并等待（signal and wait）；

  进程P等待进程直到进程Q离开管程，或者等待另一个条件；

- 唤醒并继续（signal and continue）；

  进程Q等待直到进程P离开管程或者等待另一个条件；

- 都具有合理性；

补充

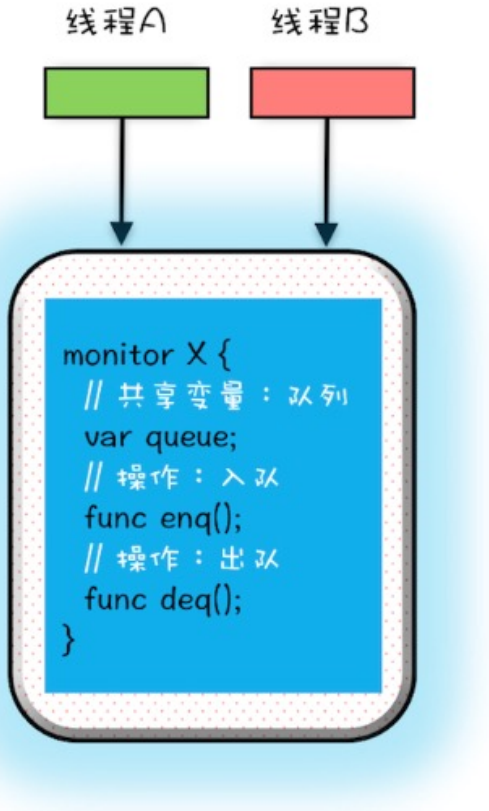
Java中的管程

Java 采用的是管程技术，synchronized 关键字及 wait()、notify()、notifyAll() 这三个方法都是管程的组成部分。而**管程和信号量是等价的，所谓等价指的是用管程能够实现信号量，也能用信号量实现管程**。但是**管程利用OOP的封装特性解决了信号量在工程实践上的复杂性问题**，因此java采用管程机制；

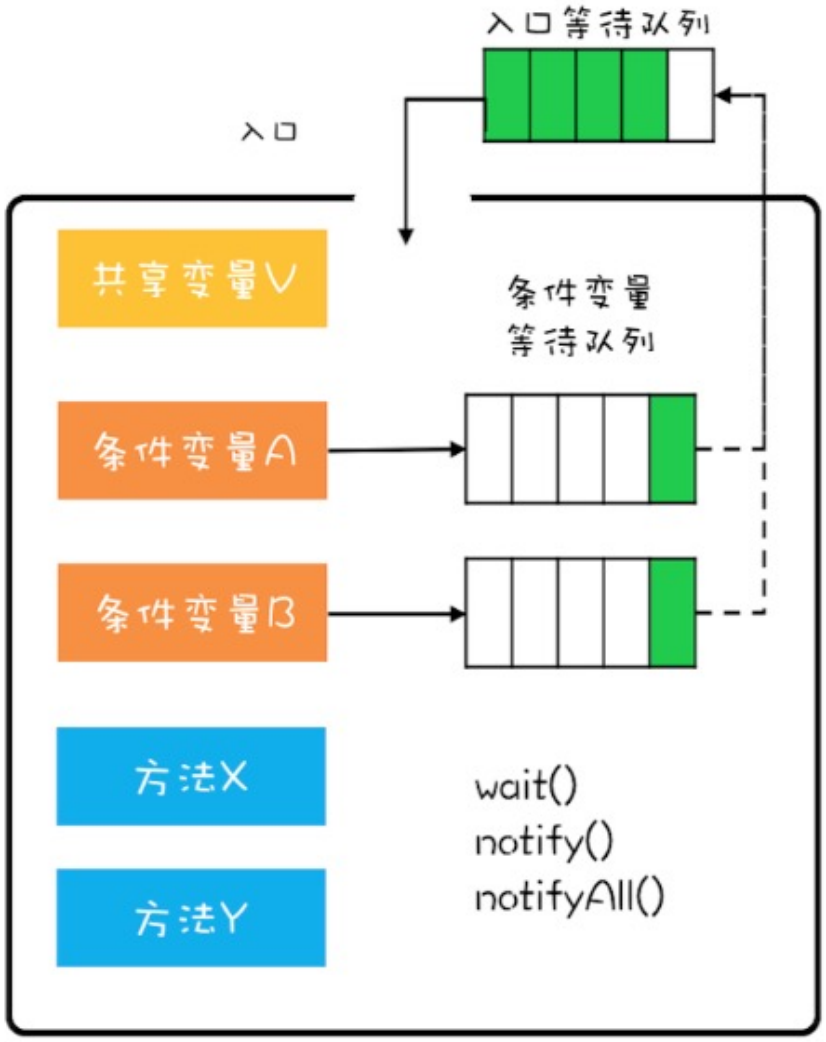
在管程的发展史上，先后出现过三种不同的管程模型，分别是：Hasen 模型、Hoare 模型和 MESA 模型。其中，现在广泛应用的是 MESA 模型，并且 Java 管程的实现参考的也是 MESA 模型。

补充

管程解决互斥问题的思路很简单，就是将共享变量及其对共享变量的操作统一封装起来。在下图中，管程 X 将共享变量 queue 这个队列和相关的操作入队 enq()、出队 deq() 都封装起来了；线程 A 和线程 B 如果想访问共享变量 queue，只能通过调用管程提供的 enq()、deq() 方法来实现；enq()、deq() 保证互斥性，只允许一个线程进入管程。不知你有没有发现，管程模型和面向对象高度契合的。估计这也是 Java 选择管程的原因吧。



补充

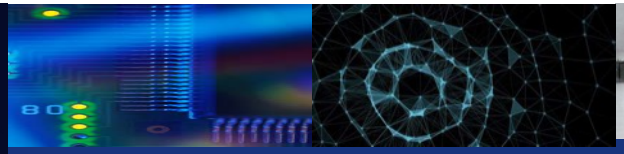


那条件变量和等待队列的作用是什么呢？其实就是解决**线程同步**问题。当进入管程的线程发现关注的变量不满足的时则进入等待队了，等关注的变量满足时则从等待队列中出来，继续执行。因此，管程的特点是**一条件、两队列**。同时我们也可以结合上面提到的入队出队例子加深一下理解。

补充

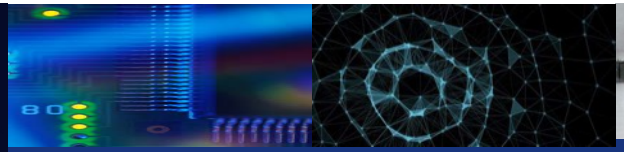
下面的代码实现的是一个阻塞队列，阻塞队列有两个操作分别是入队和出队，这两个方法都是先获取互斥锁(锁操作先忽略)，类比管程模型中的入口。

- 对于入队操作，如果队列已满，就需要等待直到队列不满，所以这里用了notFull.await();
- 对于出队操作，如果队列为空，就需要等待直到队列不空，所以就用了notEmpty.await();
- 如果入队成功，那么队列就不空了，就需要通知条件变量：队列不空notEmpty对应的等待队列;
- 如果出队成功，那就队列就不满了，就需要通知条件变量：队列不满notFull对应的等待队列



补充

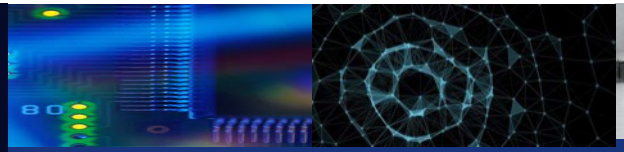
```
public class BlockedQueue<T>{
        final Lock lock = new ReentrantLock();
        // 条件变量：队列不满
        final Condition notFull = lock.newCondition();
        // 条件变量：队列不空
        final Condition notEmpty = lock.newCondition();

        // 入队
        void enq(T x) {
                lock.lock();
                try {
                        while (Stack已满){
                                // 等待队列不满
                                notFull.await();
                        }
                        // 省略入队操作...
                        // 入队后，通知可出队
                        notEmpty.signal();
                }finally {
                                lock.unlock();
                }
        }

    // 出队
        void deq(){
                lock.lock();
                try {
                        while (队列已空){
                                // 等待队列不空
                                notEmpty.await();
                        }
                        // 省略出队操作...
                        // 出队后，通知可入队
                        notFull.signal();
                }finally {
                        lock.unlock();
                }
        }
}
```



补充

有一点需要再次提醒，对于 MESA 管程来说，有一个编程范式，就是需要在一个 while 循环里面调用 wait()。**这个是 MESA 管程特有的。**

```
while(条件不满足) {
wait();
}
```



补充

Hasen 、Hoare 和 MESA 模型区别

Hasen 模型、Hoare 模型和 MESA 模型的一个核心区别就是当条件满足后，如何通知相关线程。管程要求同一时刻只允许一个线程执行，那当线程 T2 的操作使线程 T1 等待的条件满足时，T1 和 T2 究竟谁可以执行呢？

- Hasen 模型里面，要求 notify() 放在代码的最后，这样 T2 通知完 T1 后，T2 就结束了，然后 T1 再执行，这样就能保证同一时刻只有一个线程执行。
- Hoare 模型里面，T2 通知完 T1 后，T2 阻塞，T1 马上执行；等 T1 执行完，再唤醒 T2，也能保证同一时刻只有一个线程执行。但是相比 Hasen 模型，T2 多了一次阻塞唤醒操作。
- MESA 管程里面，T2 通知完 T1 后，T2 还是会接着执行，T1 并不立即执行，仅仅是从条件变量的等待队列进到入口等待队列里面。这样做的好处是 notify() 不用放到代码的最后，T2 也没有多余的阻塞唤醒操作。但是也有个副作用，就是当 T1 再次执行的时候，可能曾经满足的条件，现在已经不满足了，所以需要以循环方式检验条件变量。

经典同步问题

- 有界缓冲区同步（生产者-消费者）问题；
- 读者写者同步问题；
- 哲学家就餐问题；
- 描述Linux和Windows用于解决同步问题的工具；
- 说明如何使用POSIX和Java解决进程同步问题；

有界缓冲区同步（生产者-消费者）问题

- n个缓冲区，每个缓冲区可容纳一项
  - 信号量mutex 初始化为值1;
  - 信号量full 初始化为值0;
  - 信号量empty初始化为值n;

```
while (true) {
     . . .
   /* produce an item in next_produced */
     . . .
   wait(empty);
   wait(mutex);
     . . .
   /* add next_produced to the buffer */
     . . .
   signal(mutex);
   signal(full);
}
```

Figure 7.1 The structure of the producer process.

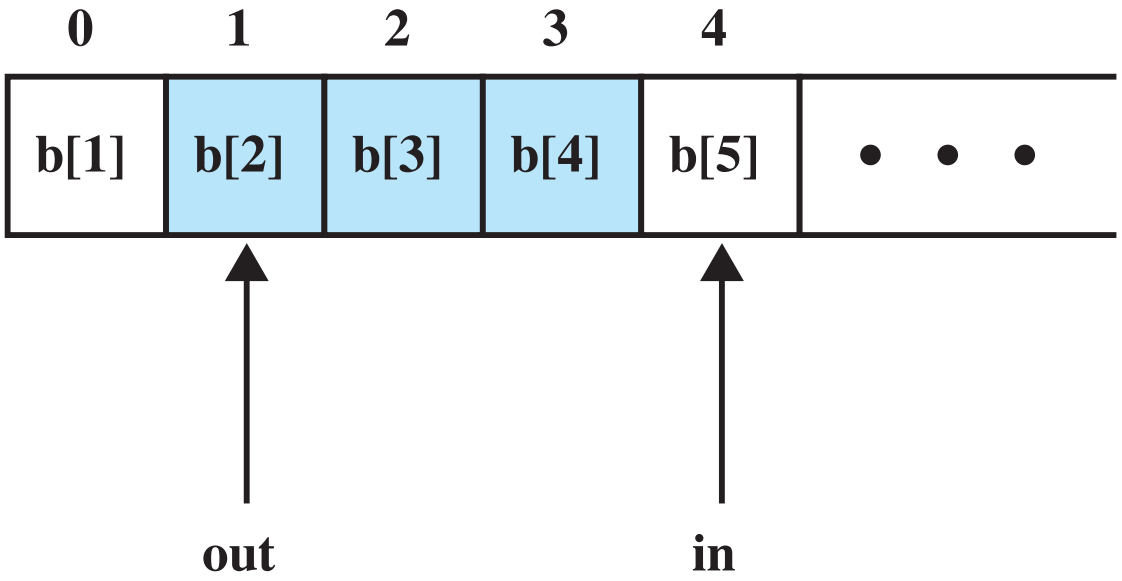
```
while (true) {
   wait(full);
   wait(mutex);
     . . .
   /* remove an item from buffer to next_consumed */
     . . .
   signal(mutex);
   signal(empty);
     . . .
   /* consume the item in next_consumed */
     . . .
}
```

Figure 7.2 The structure of the consumer process.

生产者/消费者

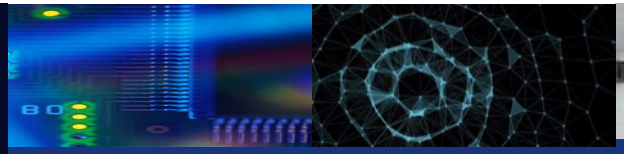
General Statement:	one or more producers are generating data and placing these in a buffer
	a single consumer is taking items out of the buffer one at a time
	only one producer or consumer may access the buffer at any one time
The Problem:	ensure that the producer can’t add data into full buffer and consumer can’t remove data from an empty buffer

无限队列



Note: shaded area indicates portion of buffer that is occupied

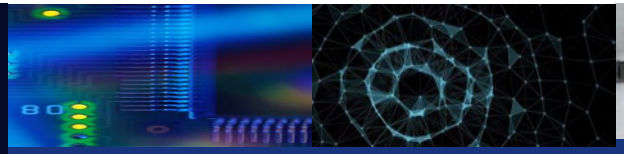
Figure 5.8 Infinite Buffer for the Producer/Consumer Problem



```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     semWaitB(delay);
     while (true) {
          semWaitB(s);
          take();
          n--;
          semSignalB(s);
          consume();
          if (n==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

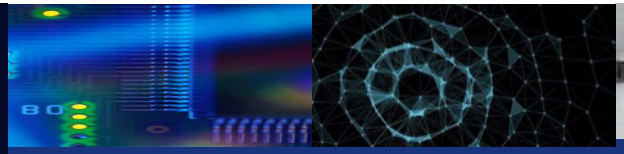
Figure 5.9

An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores



Possible Scenario for the Program of Figure 5.9

	Producer	Consumer	s	n	Delay
1			1	0	0
2	`semWaitB(s)`		0	0	0
3	`n++`		0	1	0
4	`if (n==1) (semSignalB(delay))`		0	1	1
5	`semSignalB(s)`		1	1	1
6		`semWaitB(delay)`	1	1	0
7		`semWaitB(s)`	0	1	0
8		`n--`	0	0	0
9		`semSignalB(s)`	1	0	0
10	`semWaitB(s)`		0	0	0
11	`n++`		0	1	0
12	`if (n==1) (semSignalB(delay))`		0	1	1
13	`semSignalB(s)`		1	1	1
14		`if (n==0) (semWaitB(delay))`	1	1	1
15		`semWaitB(s)`	0	1	1
16		`n--`	0	0	1
17		`semSignalB(s)`	1	0	1
18		`if (n==0) (semWaitB(delay))`	1	0	0
19		`semWaitB(s)`	0	0	0
20		`n--`	0	–1	0
21		`semSignalB(s)`	1	–1	0



```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     int m; /* a local variable */
     semWaitB(delay);
     while (true)  {
          semWaitB(s);
          take();
          n--;
          m = n;
          semSignalB(s);
          consume();
          if (m==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

Figure 5.10

A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

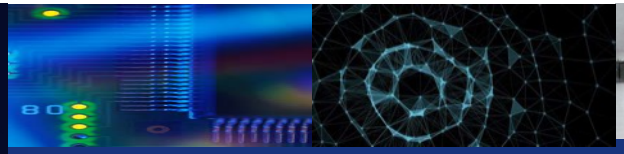
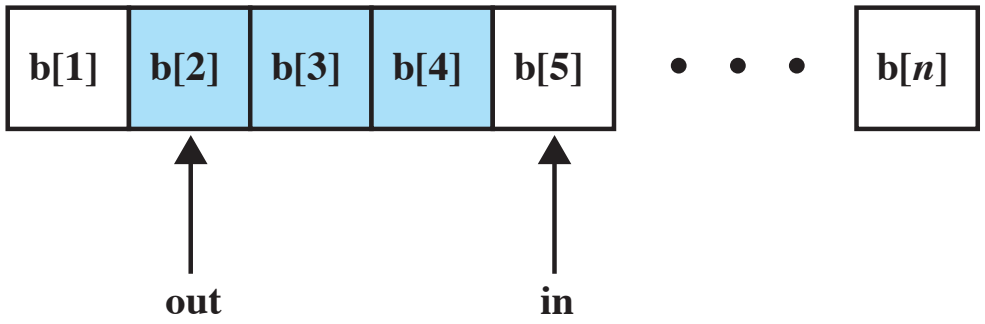


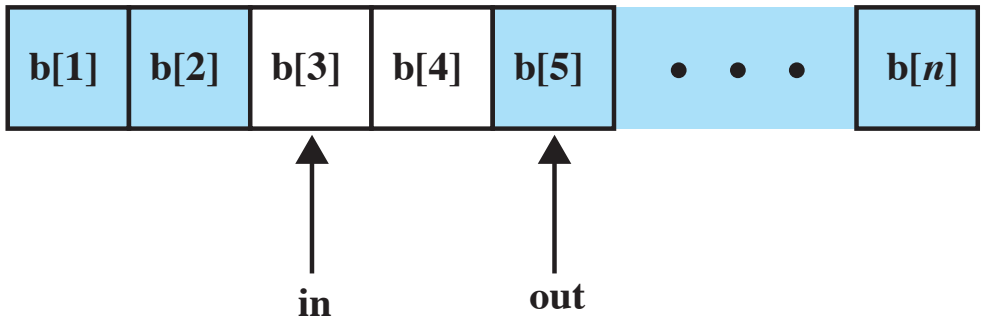
Figure 5.11

A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores

```
/* program  producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
     while (true) {
          produce();
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```



(a)



(b)

Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem

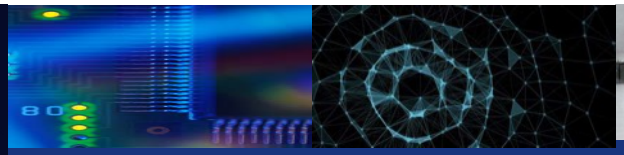


Figure 5.13

A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

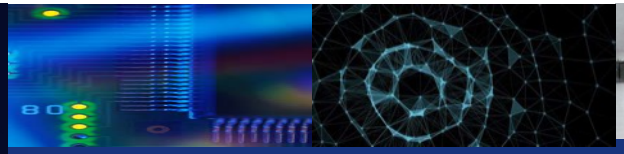
有界缓冲区管程代码

```
void append (char x)
{
     while(count == N) cwait(notfull);    /* buffer is full; avoid overflow */
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++;                                  /* one more item in buffer */
     cnotify(notempty);                       /* notify any waiting consumer */
}

void take (char x)
{
     while(count == 0) cwait(notempty); /* buffer is empty; avoid underflow */
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--;                                    /* one fewer item in buffer */
     cnotify(notfull);                        /* notify any waiting producer */
}
```

Figure 5.17 Bounded Buffer Monitor Code for Mesa Monitor

读者-写者问题

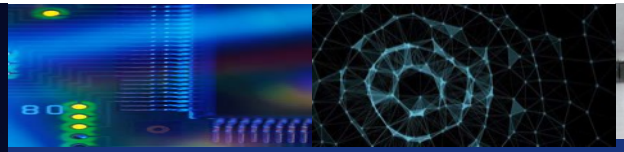
- 一个数据集在多个并发进程之间共享
  - Readers—仅读取数据集；它们不执行任何更新
  - Writer — 既能读又能写
- 问题 - 允许多个读者同时读取
  - 只有一个写者程序可以同时访问共享数据;
  - 写者在访问数据时，不允许读者访问数据;
- 读者和作者的考虑方式有几种不同——都涉及某种形式的优先权;
- 第一读者-写者问题，读者优先;
- 第二读者-写者问题，写者优先;
- 两种情况都会导致饥饿，对于第一种情况，写者饥饿；对于第二种情况，读者饥饿;



```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true) {
      semWait (x);
      readcount++;
      if (readcount == 1) semWait (wsem);
      semSignal (x);
      READUNIT();
      semWait (x);
      readcount--;
      if (readcount == 0) semSignal (wsem);
      semSignal (x);
    }
 }
void writer()
{
    while (true) {
      semWait (wsem);
      WRITEUNIT();
      semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader, writer);
}
```

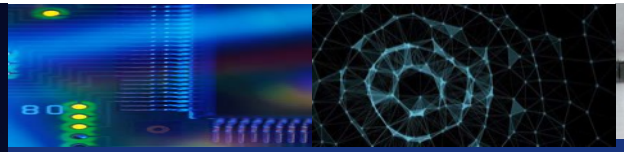
Figure 5.22

A Solution to the Readers/Writers Problem Using Semaphores: Readers Have Priority



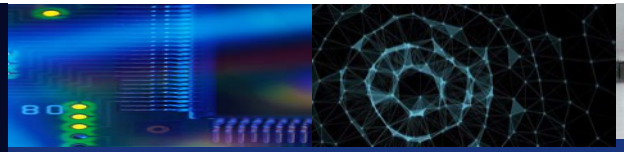
Readers only in the system	•*wsem* set •no queues
Writers only in the system	•*wsem* and *rsem* set •writers queue on *wsem*
Both readers and writers with read first	•*wsem* set by reader •*rsem* set by writer •all writers queue on *wsem* •one reader queues on *rsem* •other readers queue on *z*
Both readers and writers with write first	•*wsem* set by writer •*rsem* set by writer •writers queue on *wsem* •one reader queues on *rsem* •other readers queue on *z*

State of the Process Queues for Program of Figure 5.23



```
/*program readersandwriters*/
int  readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true) {
     semWait (z);
          semWait (rsem);
               semWait (x);
                    readcount++;
                    if (readcount == 1) semWait (wsem);
               semSignal (x);
          semSignal (rsem);
     semSignal (z);
     READUNIT();
     semWait (x);
          readcount--;
          if (readcount == 0) semSignal (wsem);
     semSignal (x);
    }
}
void writer ()
{
    while (true) {
     semWait (y);
          writecount++;
          if (writecount == 1) semWait (rsem);
     semSignal (y);
     semWait (wsem);
     WRITEUNIT();
     semSignal (wsem);
     semWait (y);
          writecount--;
          if (writecount == 0) semSignal (rsem);
     semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

A Solution to the Readers/Writers Problem Using Semaphores: Writers Have Priority



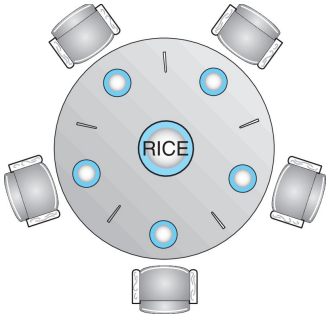
```
void reader(int i)
{
   message rmsg;
      while (true) {
         rmsg = i;
         send (readrequest, rmsg);
         receive (mbox[i], rmsg);
         READUNIT ();
         rmsg = i;
         send (finished, rmsg);
      }
 }
void writer(int j)
{
   message rmsg;
   while(true) {
      rmsg = j;
      send (writerequest, rmsg);
      receive (mbox[j], rmsg);
      WRITEUNIT ();
      rmsg = j;
      send (finished, rmsg);
   }
}
```

```
void  controller()
{
      while (true)
      {
         if (count > 0) {
            if (!empty (finished)) {
               receive (finished, msg);
               count++;
            }
            else if (!empty (writerequest)) {
               receive (writerequest, msg);
               writer_id = msg.id;
               count = count – 100;
            }
            else if (!empty (readrequest)) {
               receive (readrequest, msg);
               count--;
               send (msg.id, "OK");
            }
         }
         if (count == 0) {
            send (writer_id, "OK");
            receive (finished, msg);
            count = 100;
         }
         while (count < 0) {
            receive (finished, msg);
            count++;
         }
      }
}
```

A Solution to the Readers/Writers Problem Using Message Passing

哲学家就餐问题

- 哲学家坐在圆桌旁，中间放着一碗米饭。

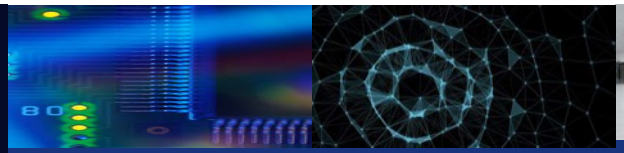


- 他们一生交替思考和进食。
- 他们不与邻居交流。
- 偶尔试着拿起2根筷子（一次一根）吃饭
  - 需要拿到2根筷子才能吃饭，然后在完成后释放
- 对于5位哲学家，共享数据
  - 一碗饭（数据集）；
  - 信号量筷子[5]初始化为1；

哲学家就餐问题：需要互斥

- **Mutual exclusion**
  - What if there is no mutual exclusion?
    - Then: while you're eating, the two men besides you will and must **steal all your chopsticks!**

- Let's proposal the following solution:
  - When you are hungry, you have to check if anyone is using the chopstick that you need.
  - If yes, you have to wait.
  - If no, **seize both chopsticks.**
  - After eating, put down all your chopsticks.



哲学家就餐问题：满足互斥

A quick question: what should be initial values?

Shared object

```
#define  N  5
semaphore chop[N];
```

Helper Functions

```
void take(int i) {
    down(&chop[i]);
}
```

```
void put(int i) {
    up(&chop[i]);
}
```

Main Function

```
1 void philosopher(int i) {
2     while (TRUE) {
3         think();
4         take(i);
5         take((i+1) % N);
6         eat();
7         put(i);
8         put((i+1) % N);
9     }
10 }
```

Section Entry

Critical Section

Section Exit

哲学家就餐问题：存在死锁

假如所有的哲学家同时饥饿，并拿起左边的筷子，当哲学家试图拿起右边的筷子时，会被永远推迟；补救措施：

- 允许4个哲学家同时坐在桌子上；
- 只有两根筷子都可用时，才能拿起它们；
- 使用非对称解决方案，单号哲学家先拿起左边的筷子，再拿起右边的筷子；而双号的哲学家先拿起右边的筷子，再拿起左边的筷子；

哲学家就餐问题：无死锁的管程解决方案

```
monitor DiningPhilosophers
{
   enum {THINKING, HUNGRY, EATING} state[5];
   condition self[5];

   void pickup(int i) {
      state[i] = HUNGRY;
      test(i);
      if (state[i] != EATING)
         self[i].wait();
   }

   void putdown(int i) {
      state[i] = THINKING;
      test((i + 4) % 5);
      test((i + 1) % 5);
   }

   void test(int i) {
      if ((state[(i + 4) % 5] != EATING) &&
       (state[i] == HUNGRY) &&
       (state[(i + 1) % 5] != EATING)) {
         state[i] = EATING;
         self[i].signal();
      }
   }

   initialization_code() {
      for (int i = 0; i < 5; i++)
         state[i] = THINKING;
    }
}
```

只有哲学家的两根筷子都可用时，才能拿起筷子；

哲学家可能饿死；

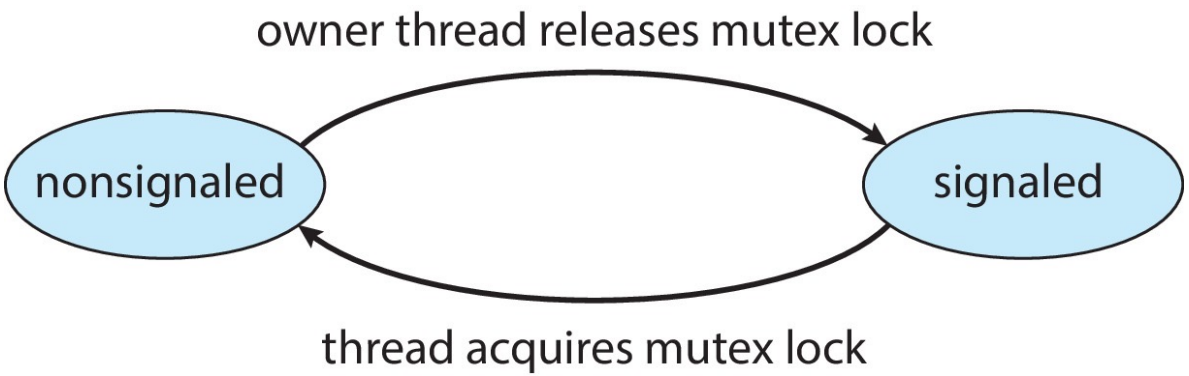
不同操作系统的同步

- Windows

- 使用中断掩码保护对单处理器系统上全局资源的访问；
- 在多处理器系统上使用自旋锁
  - 拥有自旋锁的线程永远不会被抢占
- 还提供了dispatcher对象用户区，它可以充当互斥、信号量、事件和计时器
  - 事件
    - 事件的行为非常类似于条件变量；
  - 计时器在时间过期时通知一个或多个线程
  - Dispatcher对象处于有信号状态（对象可用）或无信号状态（线程将阻塞）

内核同步-Windows

- 互斥调度程序对象



Linux同步

- Linux：
  - 在内核版本2.6之前，禁用中断以实现较短的关键部分
  - 版本2.6及更高版本，完全可抢占，内核态任务也可被抢占；
- Linux提供：
  - 信号量
  - 原子整数
  - 自旋锁
  - 读者-写者版本的自旋锁和信号量；

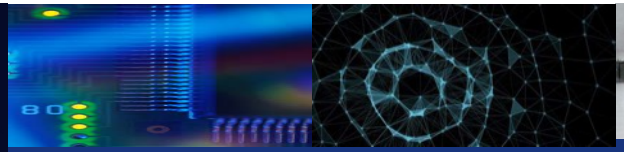
Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

- 在单CPU系统上，自旋锁被启用和禁用内核抢占取代

Single Processor	Multiple Processors
Disable kernel preemption	Acquire spin lock
Enable kernel preemption	Release spin lock

POSIX同步

- POSIXAPI提供
  - 互斥锁
  - 信号量
  - 条件变量
- 在UNIX、Linux和macOS上广泛使用



Java同步

- Java提供了丰富的同步功能：
  - Java管程
  - 可重入锁
  - 信号量
  - 条件变量

替代方案

- 事务型内存
- OpenMP
- 函数式编程语言

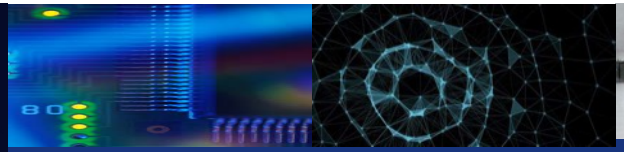
事务内存

- Consider a function update() that must be called atomically. One option is to use mutex locks:

```
void update ()
{
   acquire();

   /* modify shared data */

   release();
}
```

```
void update ()
{
   atomic {
      /* modify shared data */
   }
}
```

- A memory transaction is a sequence of read-write operations to memory that are performed atomically. A transaction can be completed by adding atomic{S} which ensure statements in S are executed atomically:



OpenMP

- OpenMP is a set of compiler directives and API that support parallel progamming.

```
void update(int value)
        {
                #pragma omp critical
                {
                        count += value
                }
        }
```

- The code contained within the #pragma omp critical directive is treated as a critical section and performed atomically.

谢谢